UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUÁREZ

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL

Bases de Datos en la Nube

Implementación y administración de bases de datos en la nube

Reporte técnico individual de la Unidad 1

Estudiante: Yhostin Manuel Ramírez González | Matrícula: 240071

Docente: Marco Antonio Ramirez Hernandez

Cuatrimestre: 7.º | Grupo: A

Fecha: 27 de octubre de 2026

ÍNDICE

Portada	1
Índice	2
Descripción de la práctica	3
A. Adquisición y configuración del VPS	3
B. Instalación de herramientas y despliegue con Docker	3
C. Puertos y conexiones remotas	5
D. Roles, usuarios y privilegios	8
E. Bases de datos y objetos relacionales	14
F. Pruebas y resultados	20
G. Control de versiones y entregables	28
Conclusiones personales	30

Descripción de la práctica

La práctica consistió en desplegar desde cero un entorno aislado de bases de datos relacionales en un VPS accesible por Internet. El objetivo fue instalar y operar PostgreSQL y MySQL con Docker Compose, administrar sus conexiones remotas, definir roles y privilegios, crear objetos relacionales y comprobar operaciones mediante consultas SQL. Se utilizaron Ubuntu 24.04.5 LTS, Git, Docker Engine y Compose, PostgreSQL 17, MySQL 8.4, pgAdmin 4, MySQL Workbench, Windows PowerShell y SQL. El alcance incluye la infraestructura, la estructura de las bases, la trazabilidad y las pruebas de usuarios y productos documentadas en este informe.

A. Adquisición y configuración del VPS

1. 1. VPS funcional y accesible. Objetivo: disponer de un servidor propio alcanzable desde Internet. Proceso: se inició sesión desde Windows PowerShell mediante SSH con la cuenta ubuntu. Resultado: se confirmó acceso administrativo al VPS en 148.113.247.81.

2. 2. Verificación de configuración. Objetivo: identificar el sistema operativo, la dirección de red y los recursos disponibles. Proceso: se revisó la información del sistema al iniciar sesión y se consultaron almacenamiento, memoria y carga. Resultado: Ubuntu 24.04.5 LTS, x86_64, kernel 6.8.0-136-generic y 71.61 GB de almacenamiento; los porcentajes de uso anotados corresponden a la sesión inicial.

3. 3. Cuenta personal del sistema. Objetivo: contar con una cuenta individual asociada al estudiante. Proceso: se creó yhostin_240071 con el nombre completo Yhostin Manuel Ramírez González. Resultado: UID/GID 1001 y directorio /home/yhostin_240071.

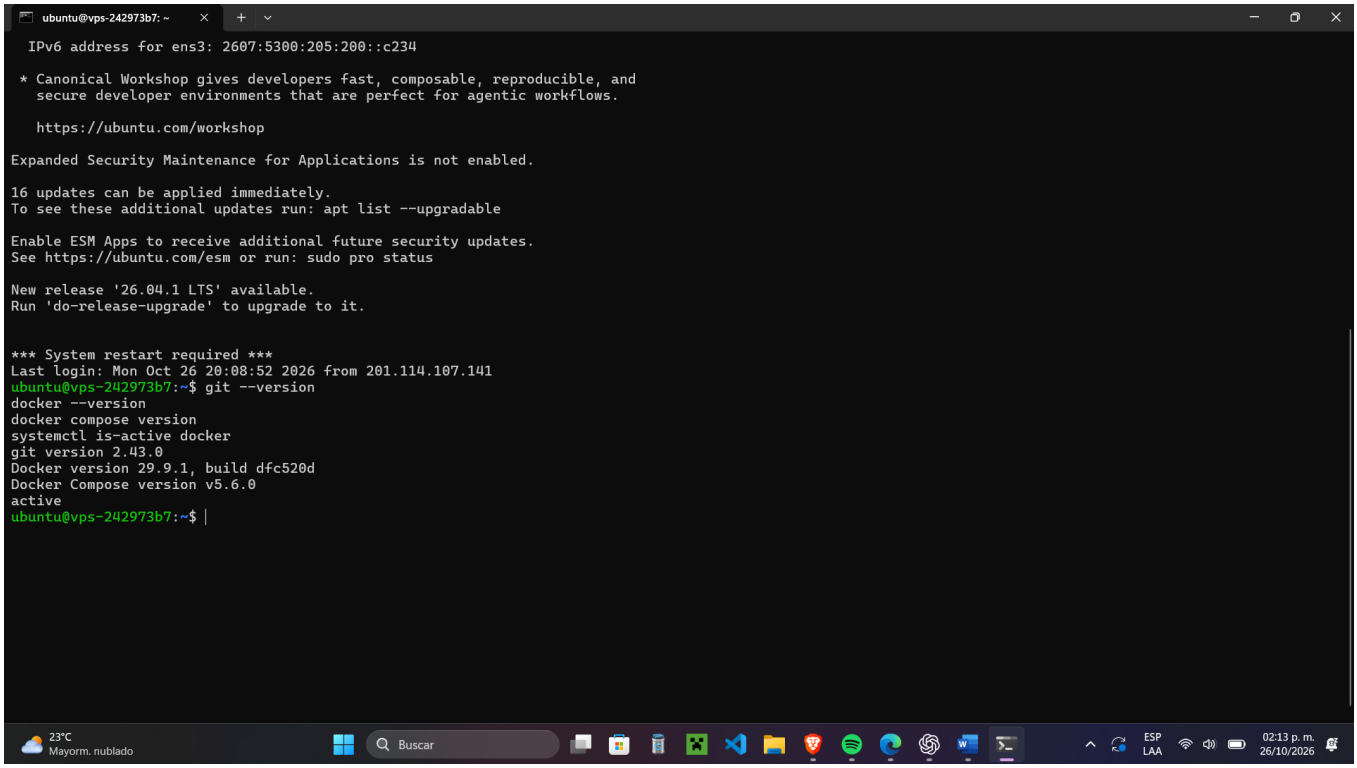
4. 4. Permisos de administración del sistema. Objetivo: distinguir la cuenta que administra el VPS de la cuenta personal. Proceso: ubuntu se utilizó con sudo para las tareas administrativas; yhostin_240071 quedó como usuario estándar del grupo users. Resultado: la evidencia no muestra que yhostin_240071 tenga sudo; la administración comprobada se hizo con ubuntu.

B. Instalación de herramientas y despliegue con Docker

5. Git instalado. Objetivo: disponer de control de versiones en el VPS. Proceso: se ejecutó git --version. Resultado: Git respondió y mostró la versión 2.43.0.

6. Docker Engine y Compose. Objetivo: instalar el entorno para desplegar los motores en contenedores. Proceso: se consultaron las versiones de Docker y Docker Compose y el estado del servicio Docker. Resultado: Docker 29.9.1, Compose v5.6.0 y servicio activo.

7. Proyecto Compose independiente. Objetivo: mantener los recursos de este ejercicio aislados de los demás proyectos del VPS. Proceso: se desplegó el proyecto desde /home/ubuntu/Manual_VPS_BDs_Relacionales con el prefijo dbnube-240071. Resultado: los contenedores y volúmenes del proyecto tienen identidad propia.



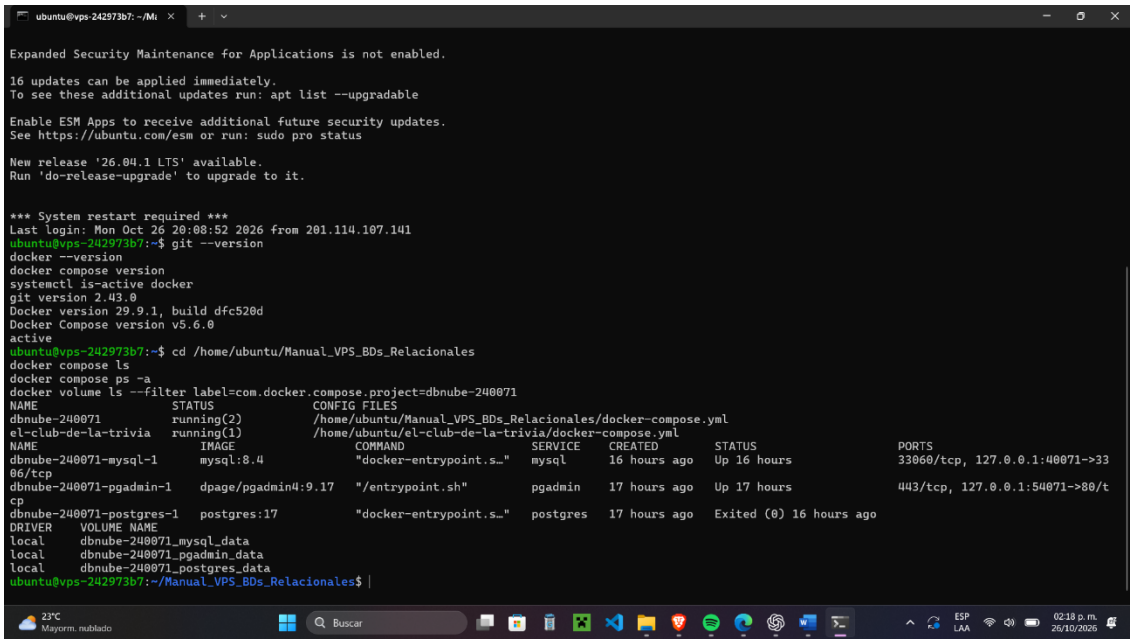
8. Servicio PostgreSQL. Objetivo: ejecutar PostgreSQL en una versión estable para el trabajo. Proceso: Docker Compose utilizó la imagen postgres:17 y se comprobó la versión desde el cliente. Resultado: el servicio respondió como PostgreSQL 17.11.

9. Servicio MySQL Server. Objetivo: ejecutar MySQL para la segunda base de datos. Proceso: Docker Compose utilizó mysql:8.4 y se consultó la conexión desde MySQL Workbench. Resultado: el servidor reportó MySQL 8.4.13.

10. pgAdmin Web. Objetivo: disponer de una interfaz gráfica para administrar PostgreSQL. Proceso: se desplegó dpage/pgadmin4:9.17 y se abrió la interfaz de pgAdmin. Resultado: la aplicación quedó disponible para registrar y consultar el servidor PostgreSQL.

11. Persistencia de datos. Objetivo: conservar la información al recrear contenedores. Proceso: el archivo Compose define volúmenes nombrados para PostgreSQL, MySQL y pgAdmin. Resultado: los servicios tienen almacenamiento persistente asociado; la evidencia de conservación tras recreación debe mantenerse junto con la prueba de Compose.

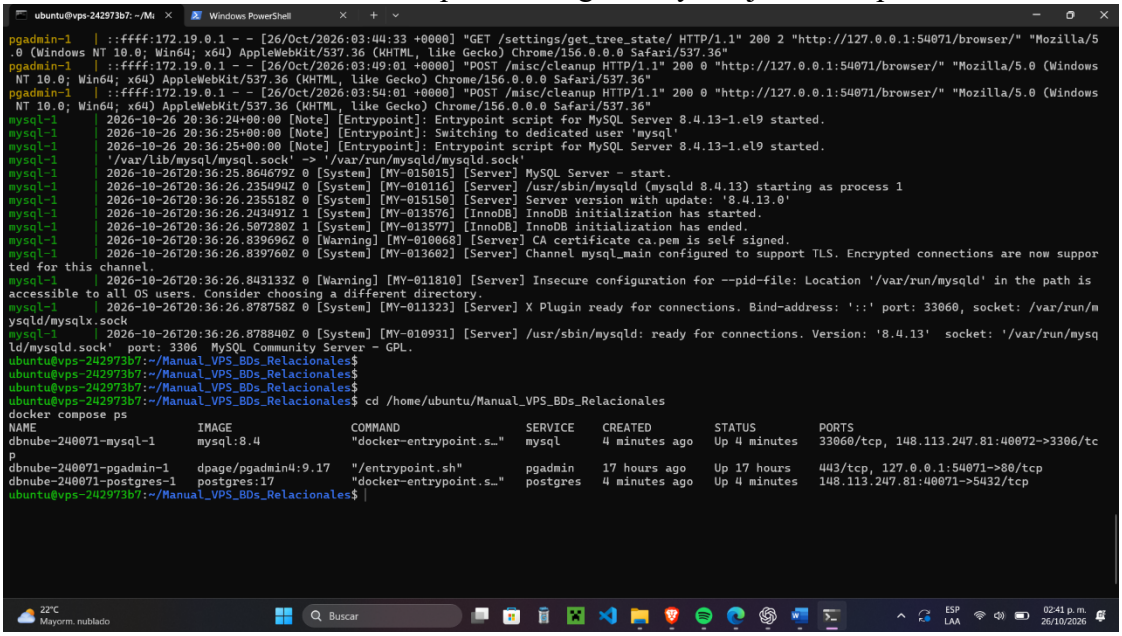
12. Estado y registros. Objetivo: comprobar que el despliegue está activo y revisar errores de inicio. Proceso: se consultaron docker compose ps y docker compose logs desde el directorio del proyecto. Resultado: las capturas muestran los contenedores en ejecución después del despliegue.



C. Configuración de puertos y conexiones remotas

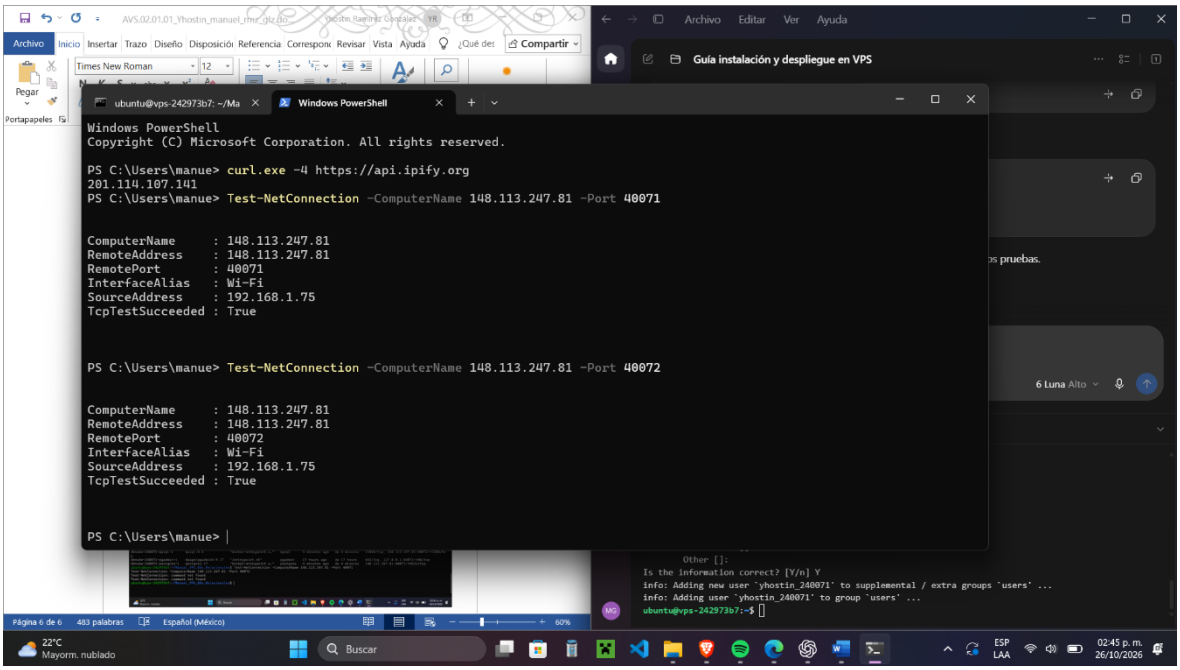
13. Puerto de PostgreSQL. Objetivo: publicar PostgreSQL en el puerto indicado por la matrícula. Proceso: se asignó el puerto TCP 40071, correspondiente a 240071. Resultado: pgAdmin se conectó al VPS por 148.113.247.81:40071. 14. Puerto de MySQL. Objetivo: publicar MySQL en la matrícula más uno. Proceso: se asignó el puerto TCP 40072. Resultado: MySQL Workbench se conectó al VPS por 148.113.247.81:40072.

15. Puertos válidos y sin conflicto. Objetivo: usar puertos distintos para ambos motores y evitar interferencia con otros servicios. Proceso: se revisaron los puertos asignados y se ejecutaron pruebas de conectividad

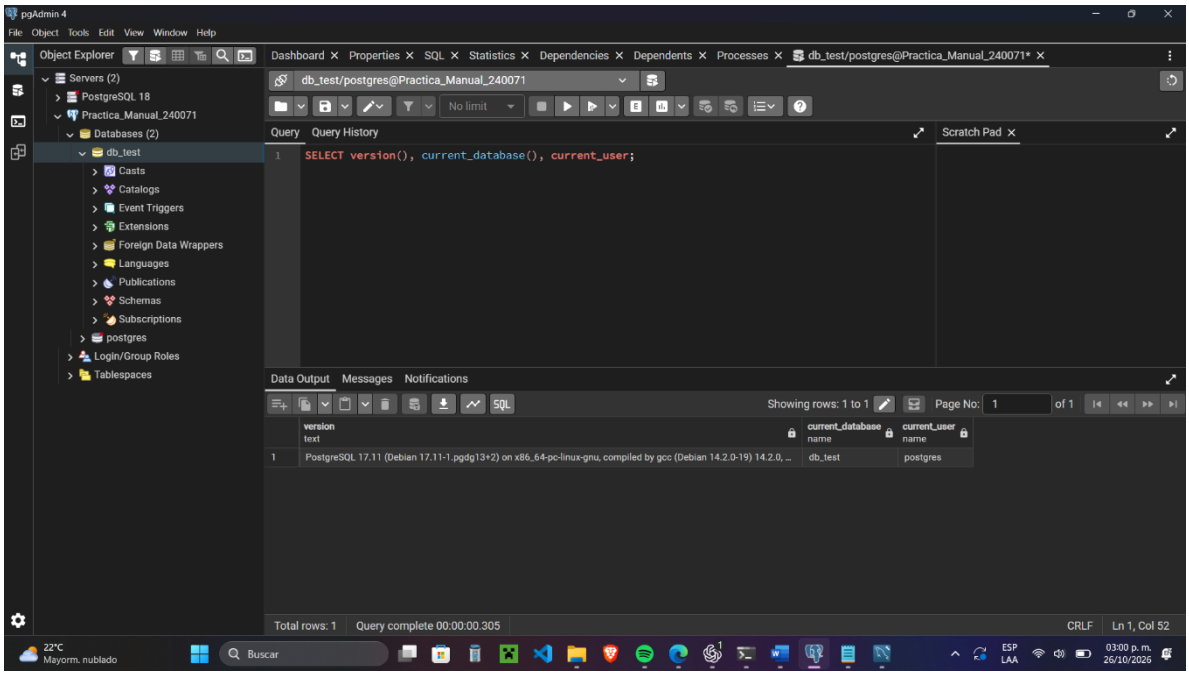


desde PowerShell. Resultado: 40071 y 40072 respondieron; la evidencia de la otra computadora muestra conexión TCP exitosa tras incluir su IP en la lista autorizada.

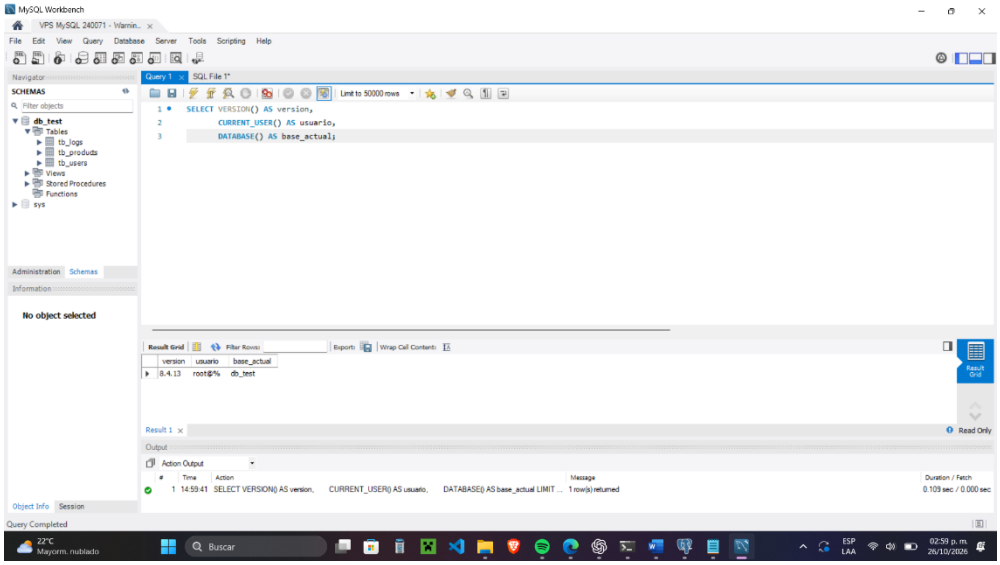
16. Conexiones remotas autorizadas. Objetivo: permitir el acceso desde clientes externos con autenticación y filtrado de red. Proceso: se habilitaron los puertos de cada motor y se limitaron por IP las reglas de la cadena DOCKER-USER. Resultado: los clientes autorizados pueden alcanzar los puertos; la política descarta tráfico de otras IP para esos puertos.



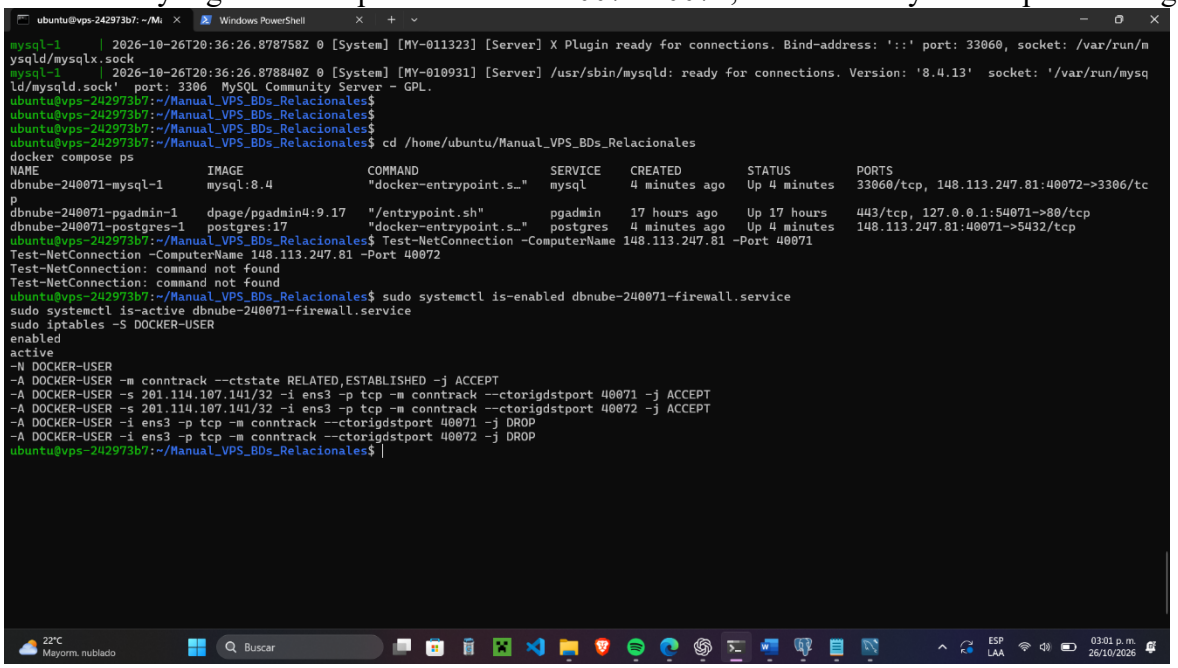
17. Conexión gráfica a PostgreSQL. Objetivo: demostrar acceso remoto al motor con una cuenta administradora. Proceso: pgAdmin se conectó a 148.113.247.81:40071, base db_test, usuario postgres; se ejecutó una consulta de identificación. Resultado: PostgreSQL 17.11, db_test y postgres aparecieron en el resultado.



18. Conexión gráfica a MySQL. Objetivo: demostrar acceso remoto a MySQL con una cuenta administradora. Proceso: MySQL Workbench se conectó a 148.113.247.81:40072 y consultó versión, usuario y esquema. Resultado: la conexión a db_test funcionó y el servidor reportó MySQL 8.4.13.



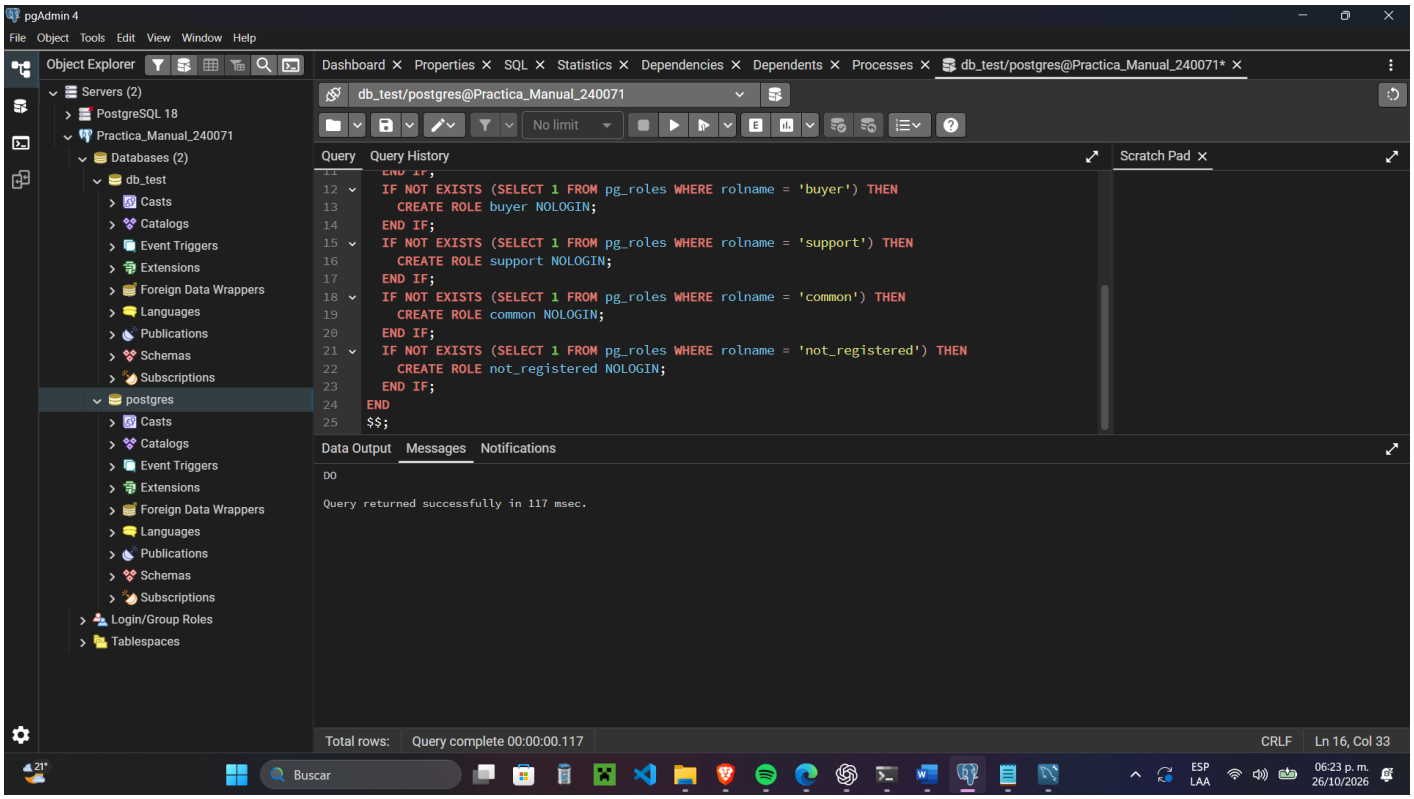
19. Firewall y restricción de acceso. Objetivo: permitir sólo clientes autorizados a los puertos de bases de datos. Proceso: se configuró DOCKER-USER con aceptación de conexiones establecidas, reglas ACCEPT para IP autorizadas y reglas DROP para el resto en 40071/40072; un servicio systemd aplica las reglas al



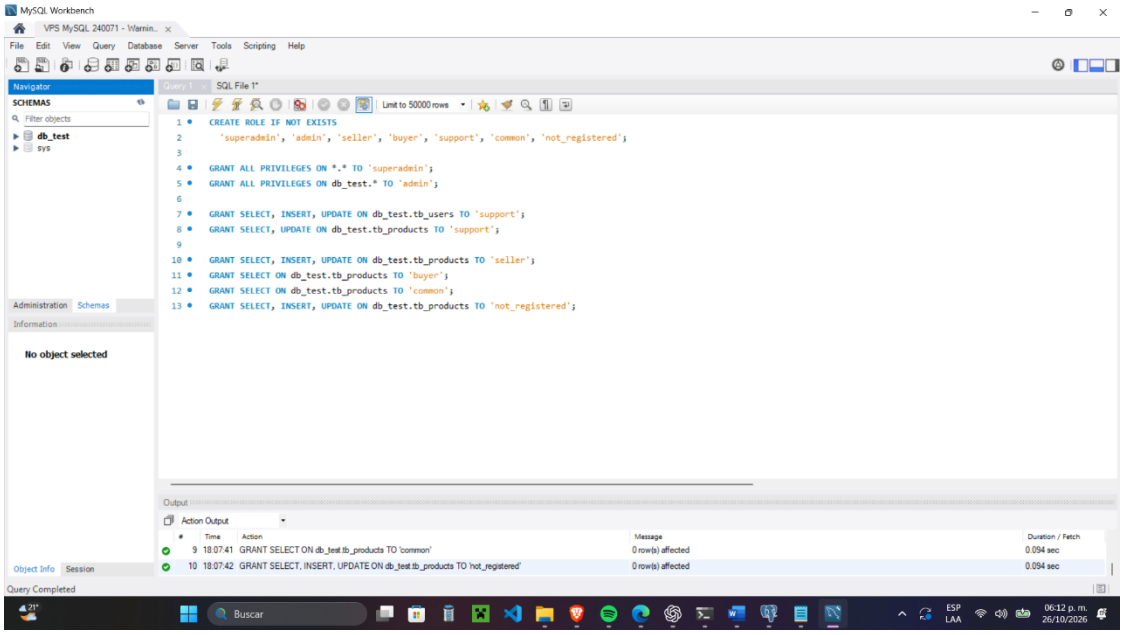
iniciar. Resultado: las reglas aparecen activas y el servicio figura habilitado. La IP de cada cliente externo debe actualizarse si cambia.

D. Administración de roles, usuarios y privilegios

20. Roles de PostgreSQL. Objetivo: crear los siete perfiles de negocio solicitados. Proceso: se crearon superadmin, admin, seller, buyer, support, common y not_registered como roles grupales sin LOGIN. Resultado: pg_roles mostró los siete roles con rol de inicio de sesión desactivado.

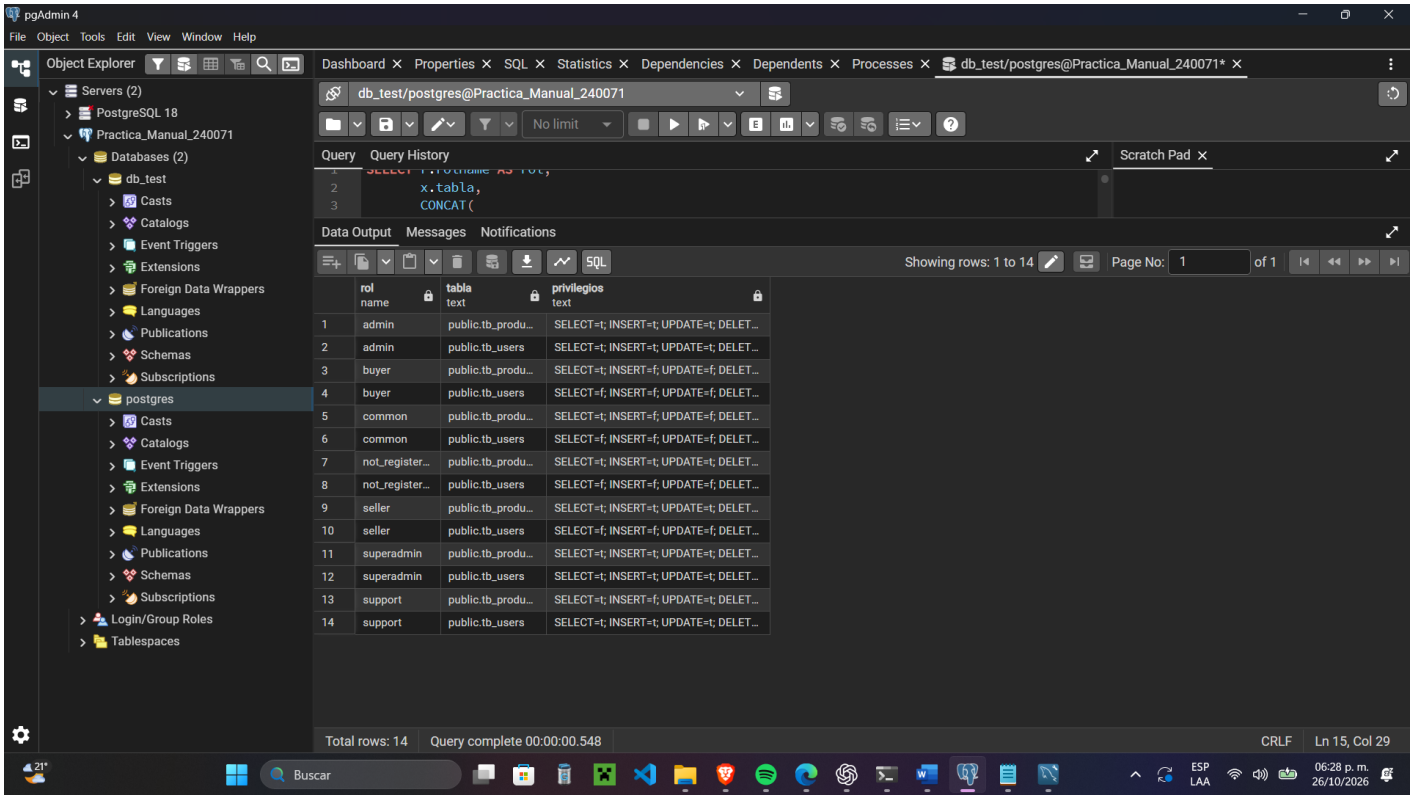


21. Roles de MySQL. Objetivo: disponer de los siete perfiles equivalentes en MySQL. Proceso: se crearon superadmin, admin, seller, buyer, support, common y not_registered. Resultado: la consulta del catálogo de

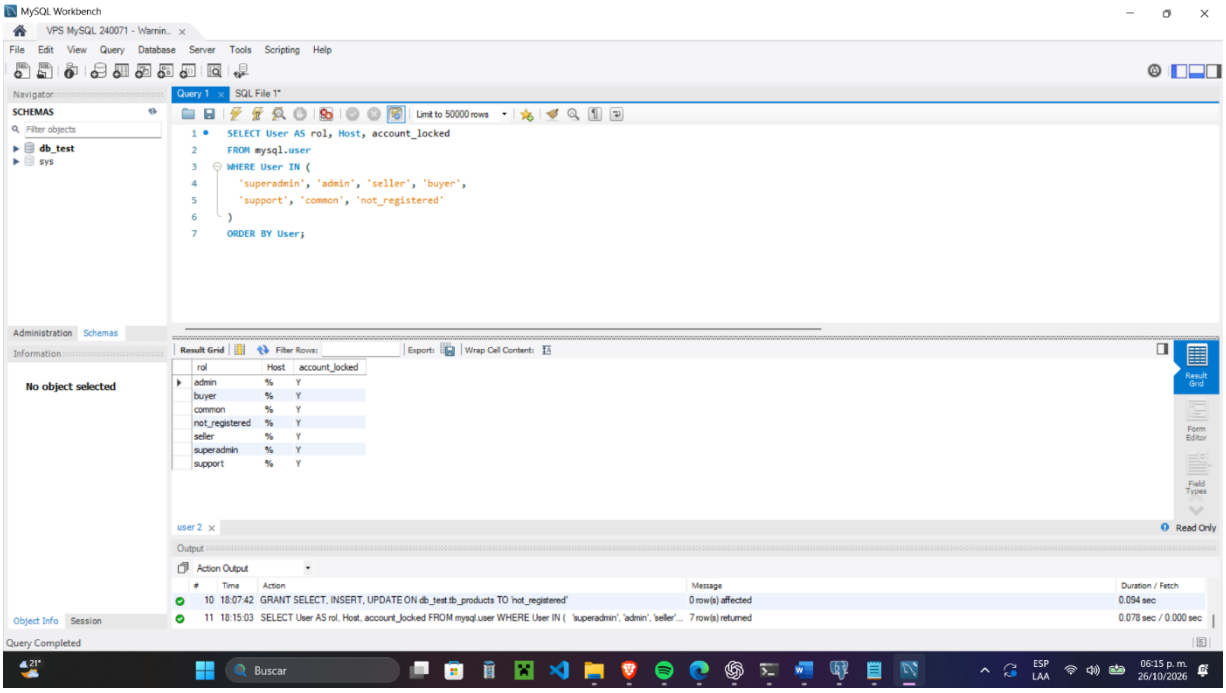


roles devolvió los perfiles requeridos.

22. Privilegios de PostgreSQL. Objetivo: limitar las operaciones permitidas a cada perfil de acuerdo con las reglas de la práctica. Proceso: se prepararon GRANT para administración, support, seller y perfiles de consulta sobre public, tablas y secuencias. Resultado: una captura inicial muestra que el GRANT a tb_users falló porque esa tabla aún no existía en ese momento; repetir y guardar la verificación de privilegios sobre las tablas ya creadas antes de dar este criterio por completo.

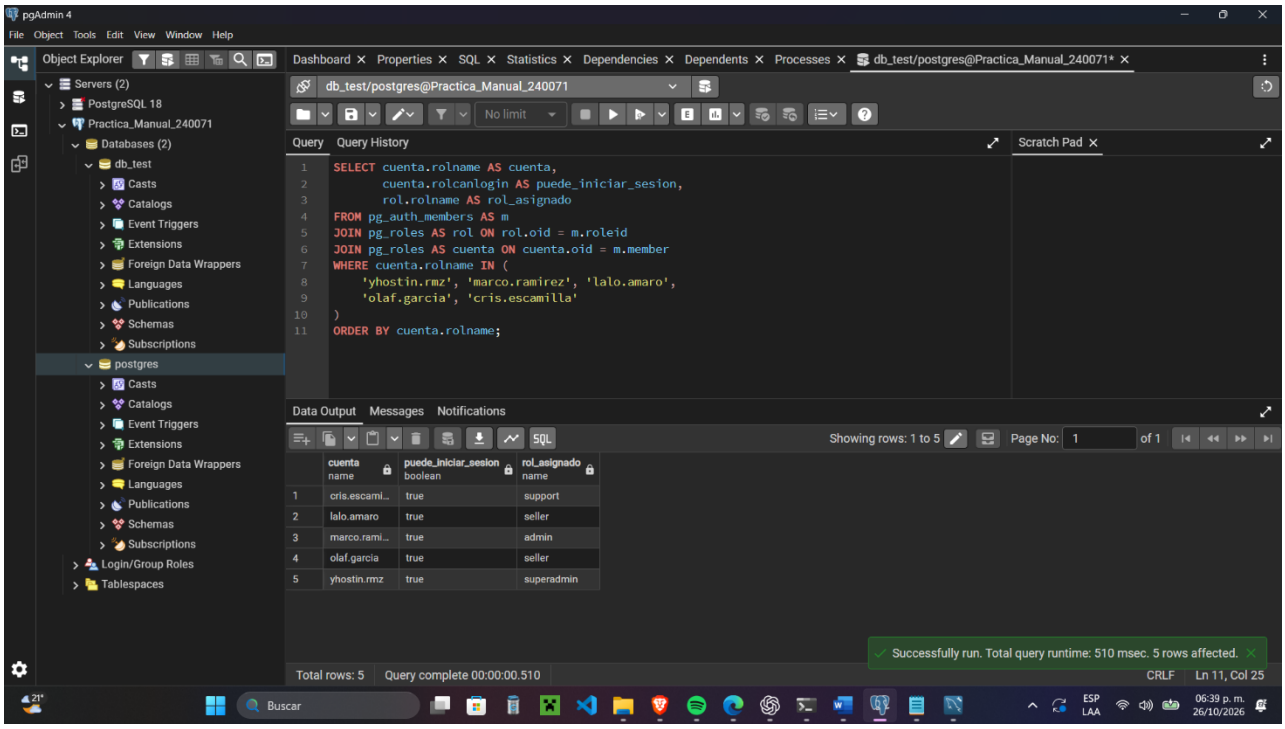


23. Privilegios de MySQL. Objetivo: aplicar permisos diferenciados por rol en db_test. Proceso: se asignaron y consultaron GRANT para superadmin, admin, seller, support, buyer, common y not_registered. Resultado:

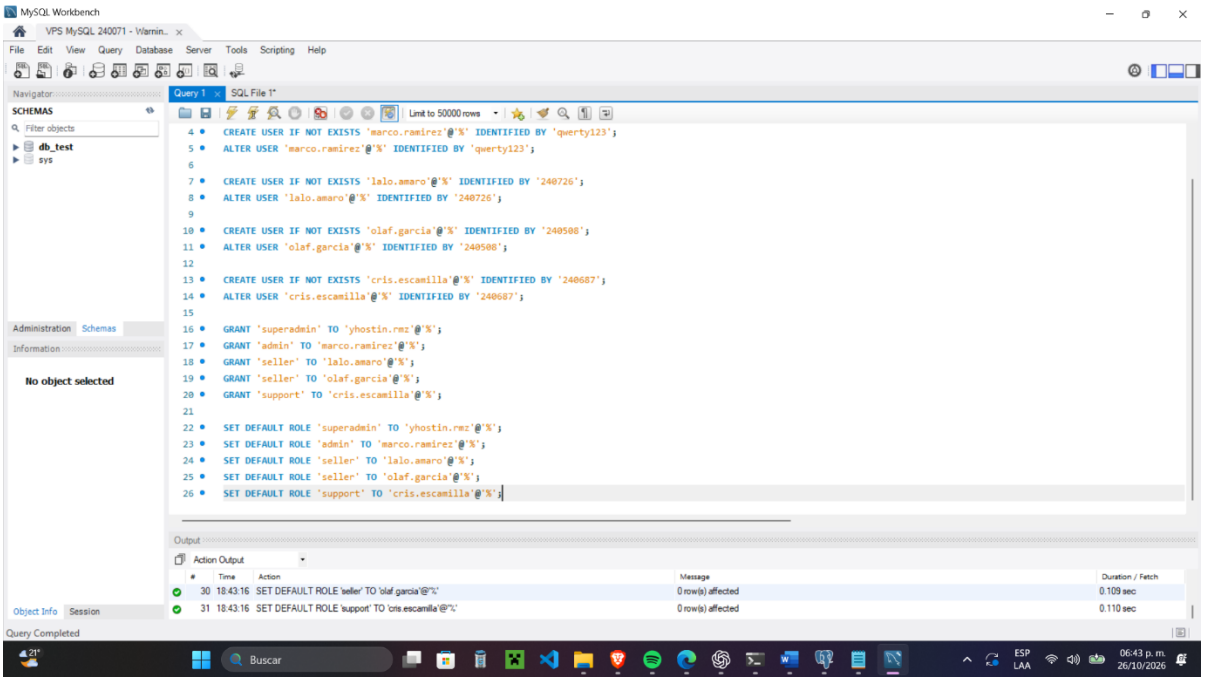


las consultas documentan administración, permisos de soporte sobre usuarios/productos, permisos de seller sobre productos y acceso limitado a productos para not_registered; conservar la consulta SHOW GRANTS como evidencia.

24. Cuentas personales en PostgreSQL. Objetivo: asociar integrantes reales con los perfiles requeridos. Proceso: se crearon yhostin.rmz, marco.ramirez, lalo.amaro, olaf.garcia y cris.escamilla y se les concedieron los roles correspondientes. Resultado: las cinco cuentas aparecen vinculadas a superadmin, admin, seller (dos cuentas) y support; la autenticación de cris.escamilla aún requiere prueba exitosa.

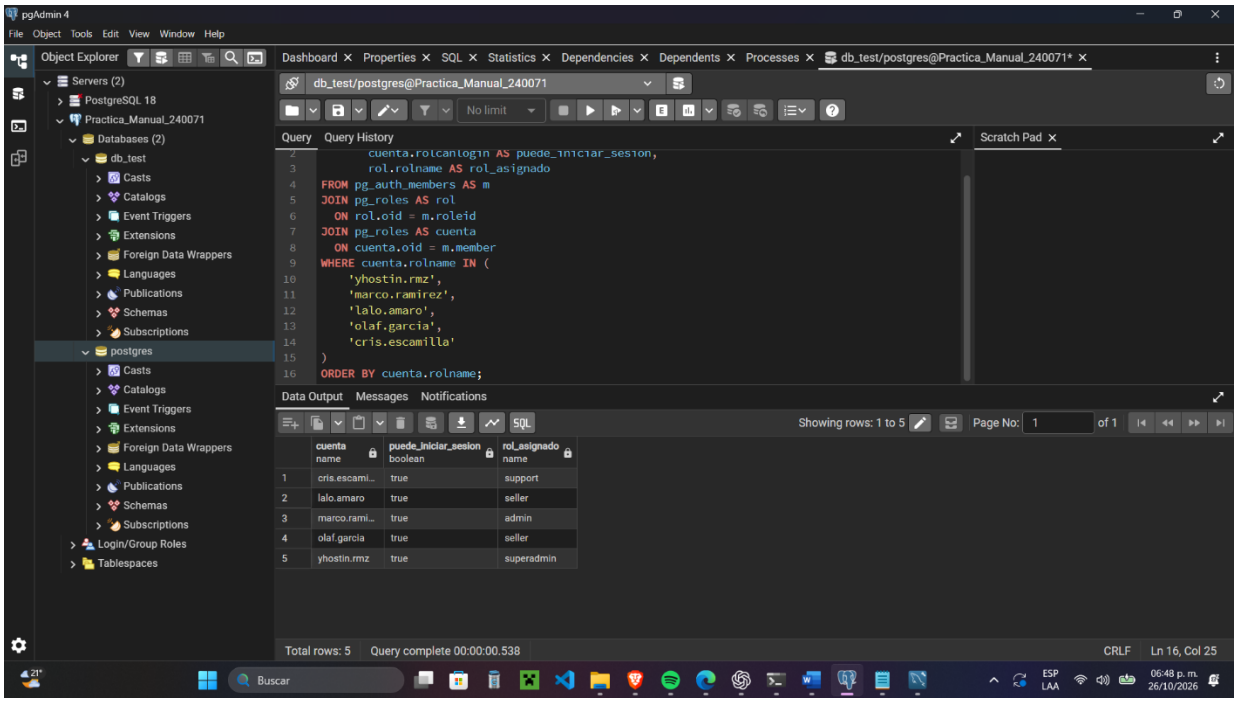


25. Cuentas personales en MySQL. Objetivo: crear las cinco cuentas personales del ejercicio en MySQL. Proceso: se crearon las cuentas y se concedieron superadmin, admin, dos seller y support. Resultado: las



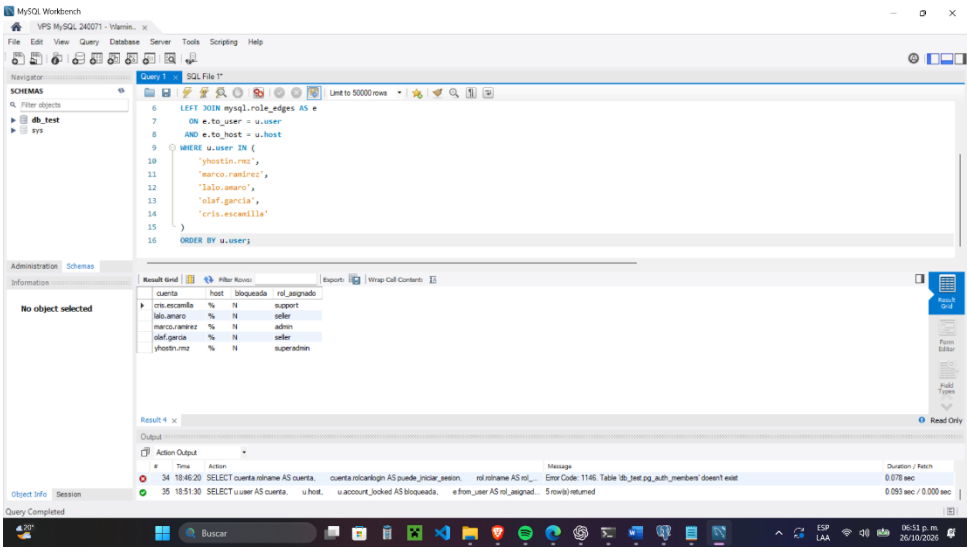
consultas del catálogo muestran las cuentas y membresías registradas.

26. Verificación de roles en PostgreSQL. Objetivo: comprobar las membresías de las cuentas personales.



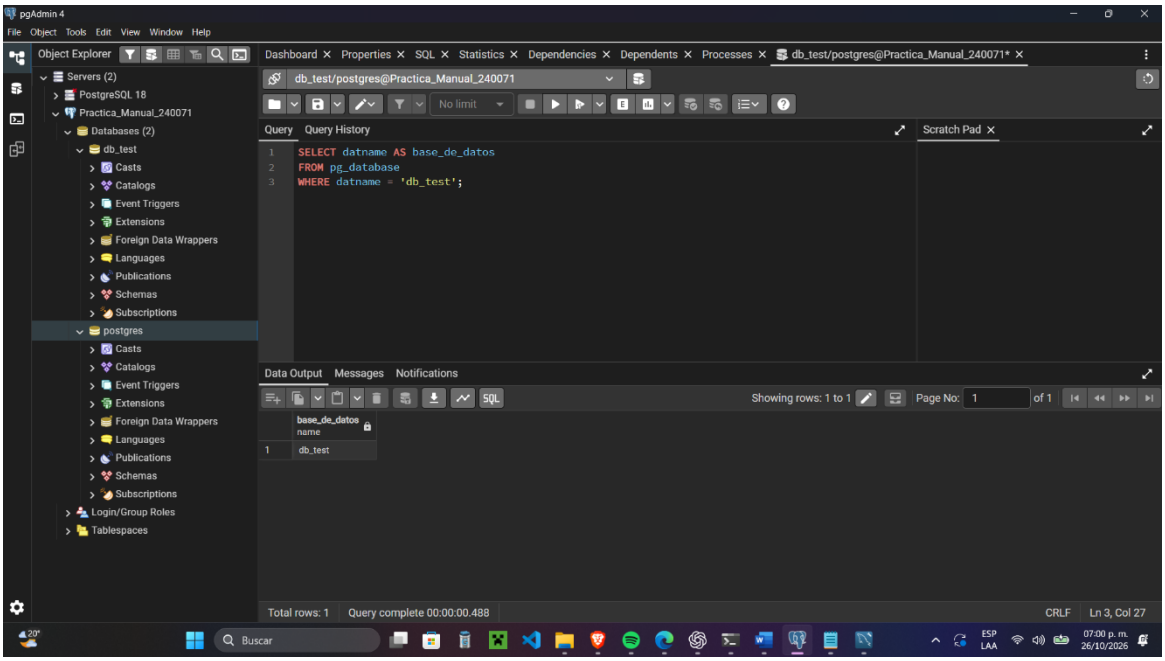
Proceso: se consultaron pg_roles y las relaciones de membresía para contrastar cuenta y rol. Resultado: la evidencia disponible muestra el catálogo; añadir una consulta que presente juntas las cinco cuentas y sus roles si no aparece completa en la captura.

27. Verificación de roles en MySQL. Objetivo: demostrar qué rol recibió cada cuenta. Proceso: se consultó mysql.role_edges y la asignación de rol predeterminado. Resultado: aparecen yhostin.rmz/superadmin, marco.ramirez/admin, lalo.amaro y olaf.garcia/seller, cris.escamilla/support y usuario_no_registrado/not_registered.



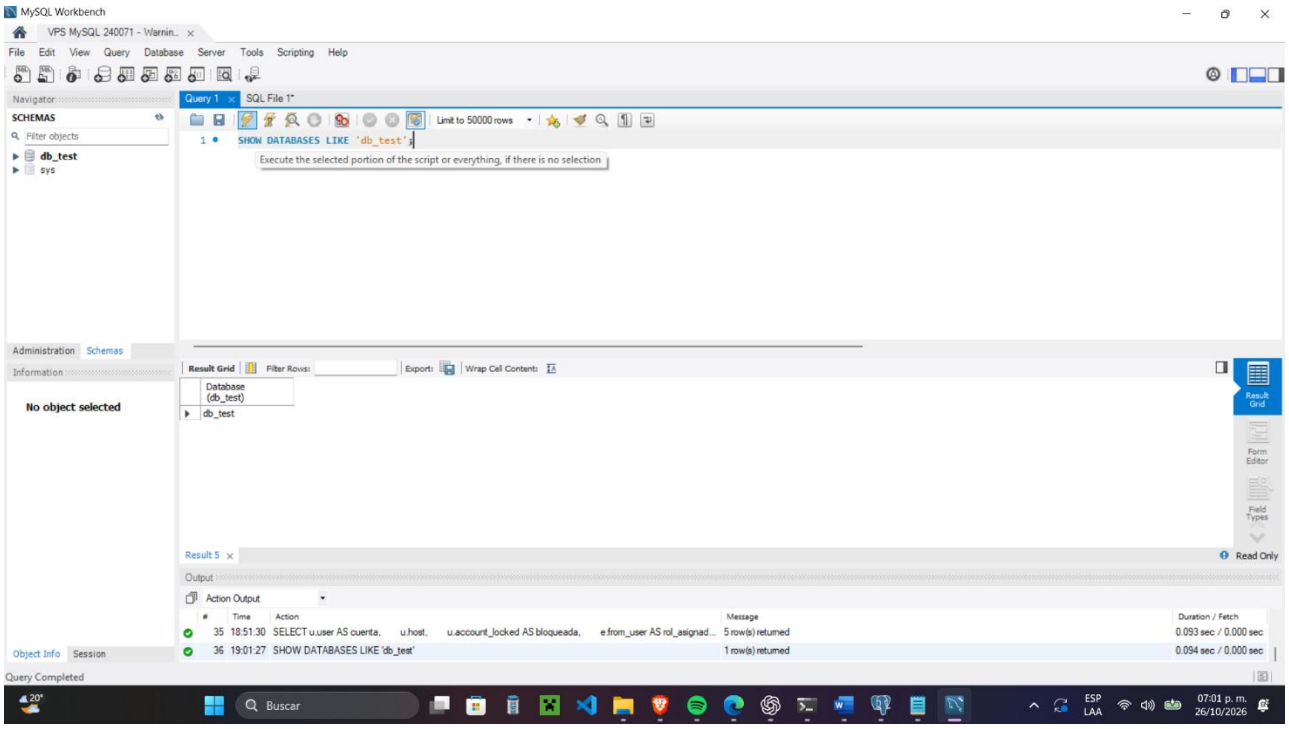
E. Creación de bases de datos y objetos relacionales

28. Base de datos PostgreSQL. Objetivo: crear el contenedor lógico solicitado para los objetos del ejercicio.

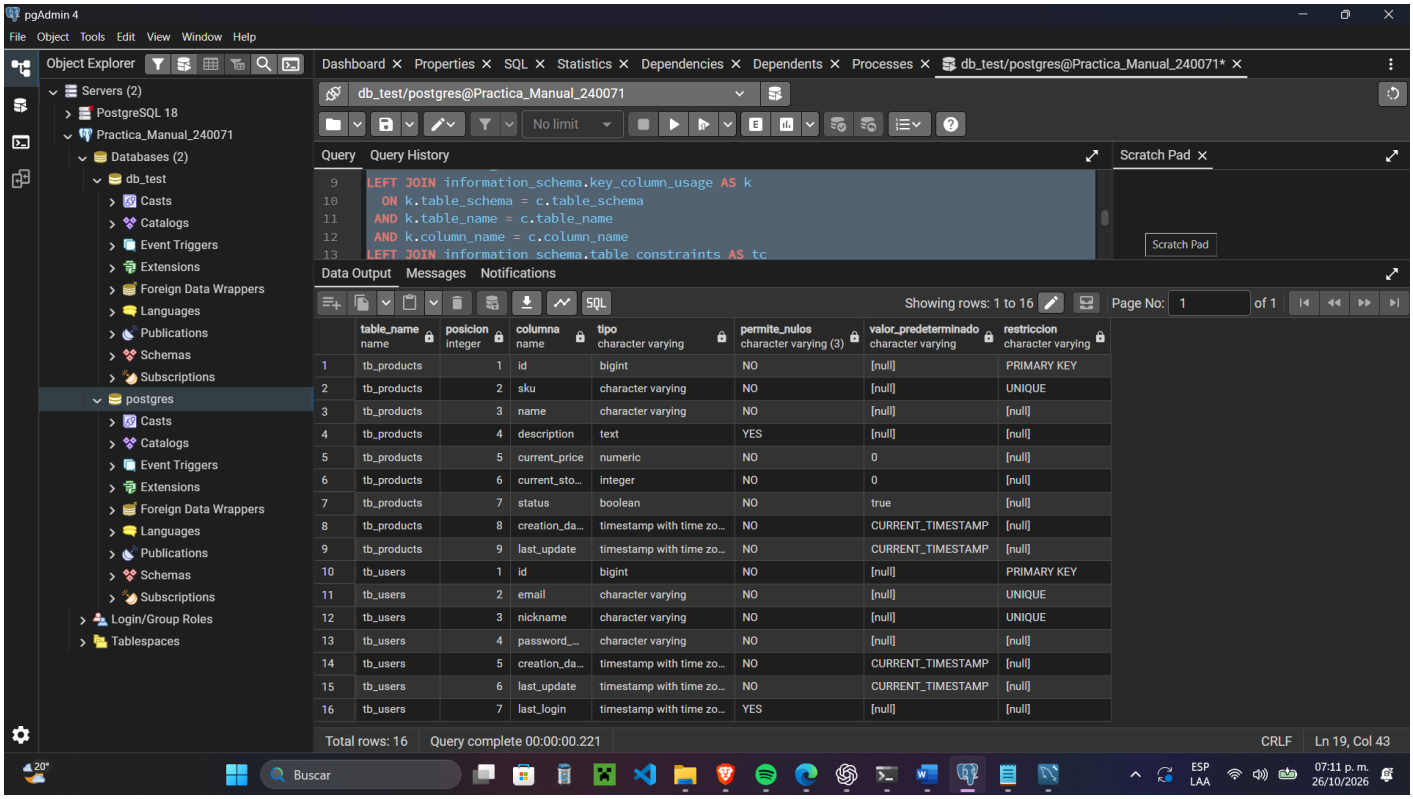


Proceso: se creó db_test y se consultó current_database() desde pgAdmin. Resultado: la sesión indicó db_test como base actual.

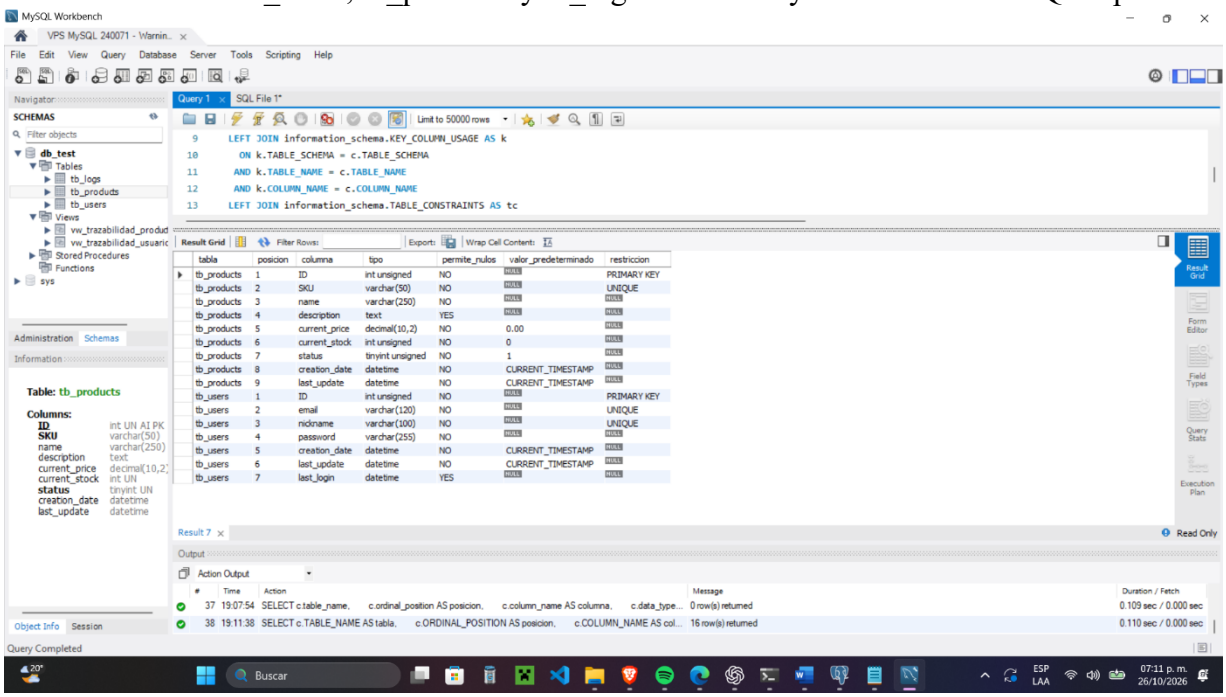
29. Base de datos MySQL. Objetivo: crear la base equivalente en MySQL. Proceso: se creó db_test y se seleccionó en Workbench. Resultado: DATABASE() confirmó db_test como esquema activo.



30. Tablas de PostgreSQL. Objetivo: construir las entidades de usuarios y productos con sus restricciones. Proceso: se crearon tb_users y tb_products con claves primarias, campos definidos y restricciones de unicidad para identificadores de negocio. Resultado: el catálogo mostró columnas de usuarios y productos, incluyendo password_hash, SKU, precio, inventario, estado y marcas de tiempo.

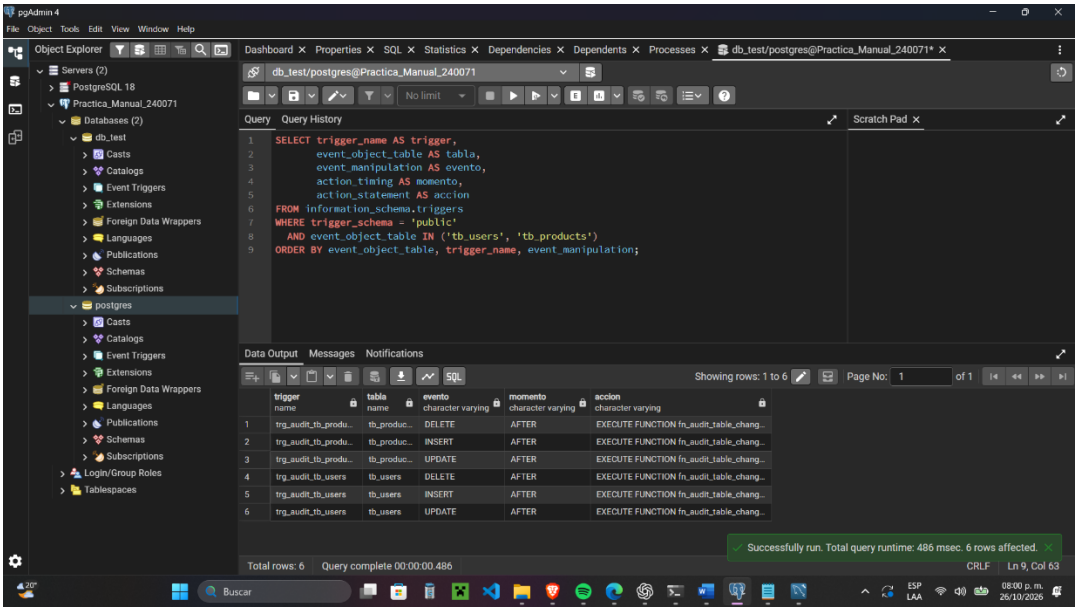


31. Tablas de MySQL. Objetivo: crear la estructura equivalente en MySQL y una tabla para auditoría. Proceso: se definieron tb_users, tb_products y tb_logs con claves y restricciones UNIQUE para email,



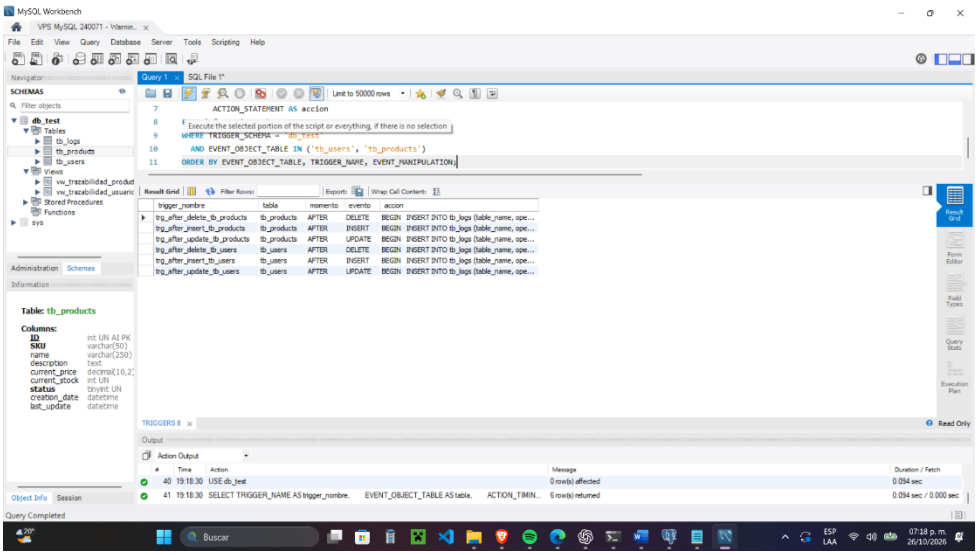
nickname y SKU. Resultado: Workbench mostró las tablas y sus columnas; verificar los nombres/tipos contra el esquema de clase antes de la entrega final.

32. Triggers de PostgreSQL. Objetivo: registrar cambios de usuarios y productos automáticamente. Proceso:



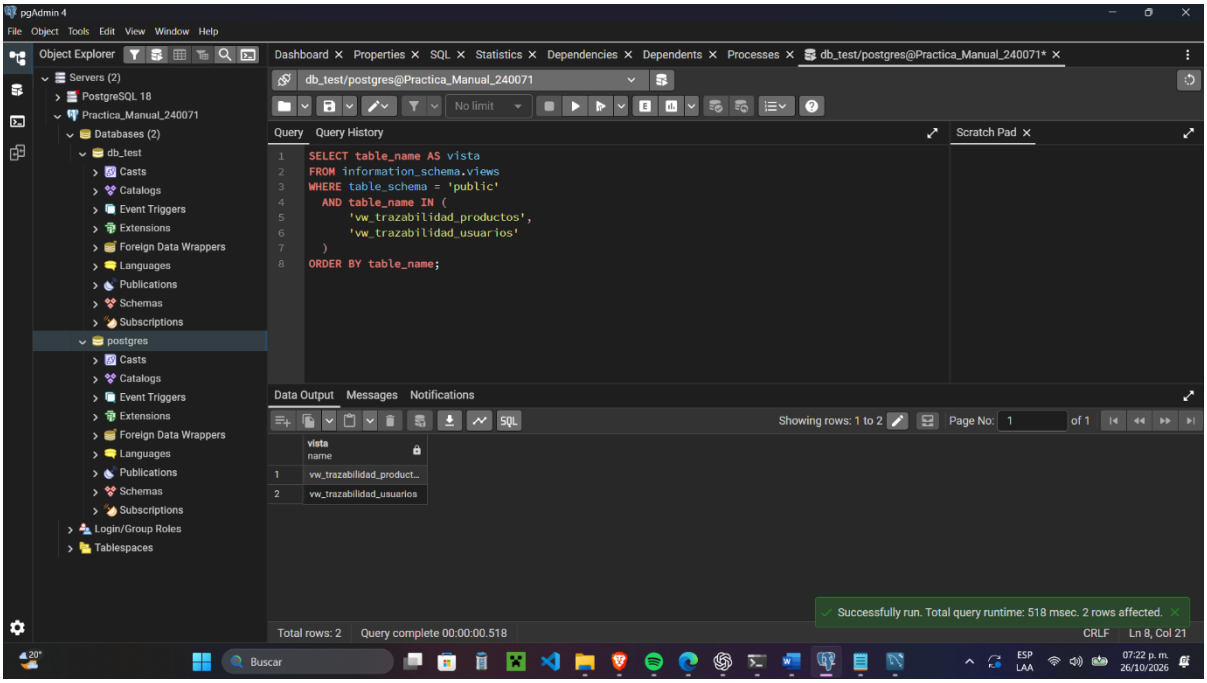
se instaló una función de auditoría y triggers AFTER INSERT, UPDATE y DELETE para tb_users y tb_products. Resultado: el catálogo muestra seis eventos asociados a las dos tablas y las operaciones solicitadas.

33. Triggers de MySQL. Objetivo: registrar en MySQL las altas, modificaciones y bajas. Proceso: se crearon

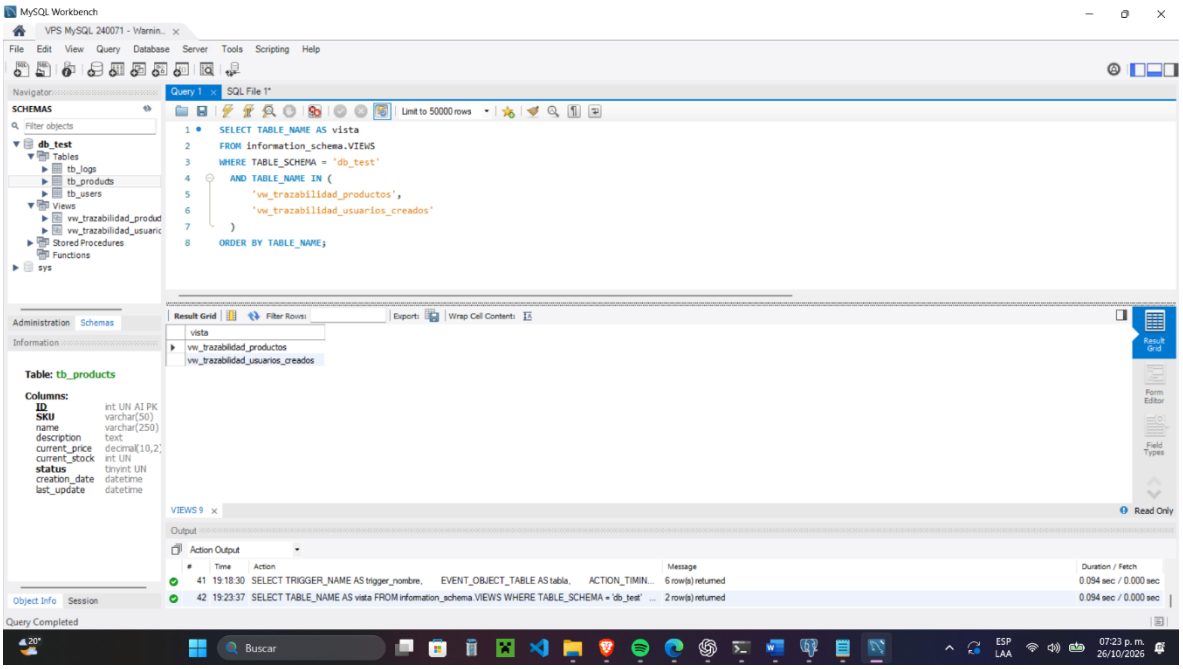


triggers para INSERT, UPDATE y DELETE sobre tb_users y tb_products que escriben en tb_logs. Resultado: las consultas al catálogo muestran los triggers y los registros de prueba aparecen en la bitácora.

34. Vista de trazabilidad en PostgreSQL. Objetivo: facilitar la consulta de eventos registrados. Proceso: se creó la vista de trazabilidad y se consultó su definición/resultado en pgAdmin. Resultado: la vista quedó en el catálogo; la primera consulta de comprobación devolvió cero filas antes de generar nuevos eventos, por lo que se añadió después una operación de prueba y su revisión en tb_logs.

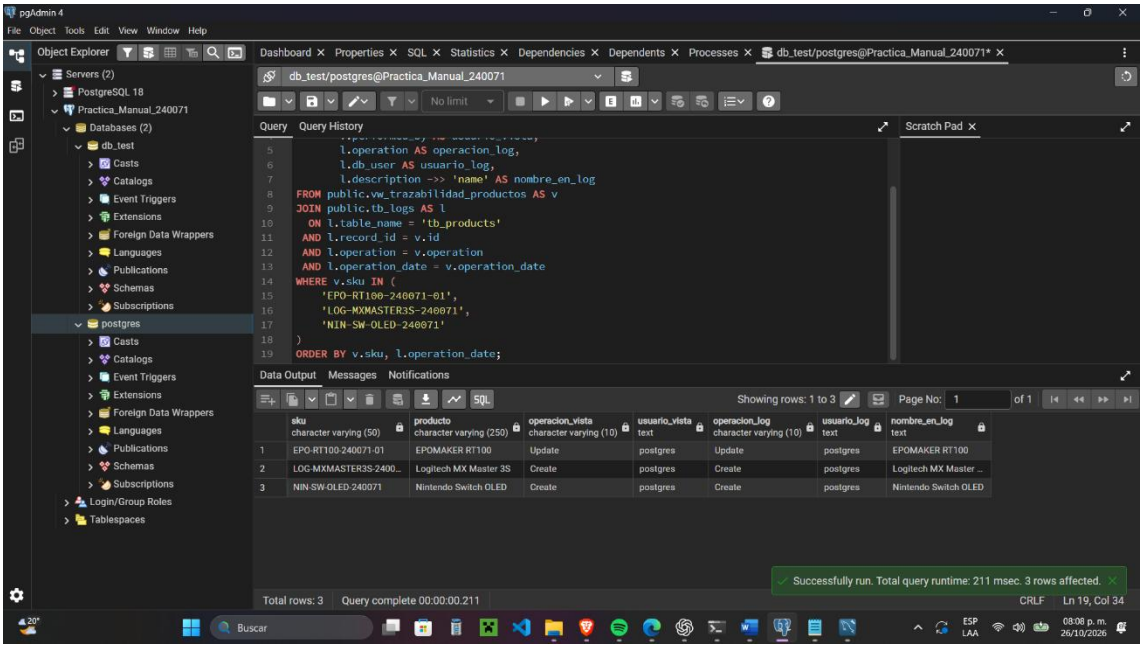


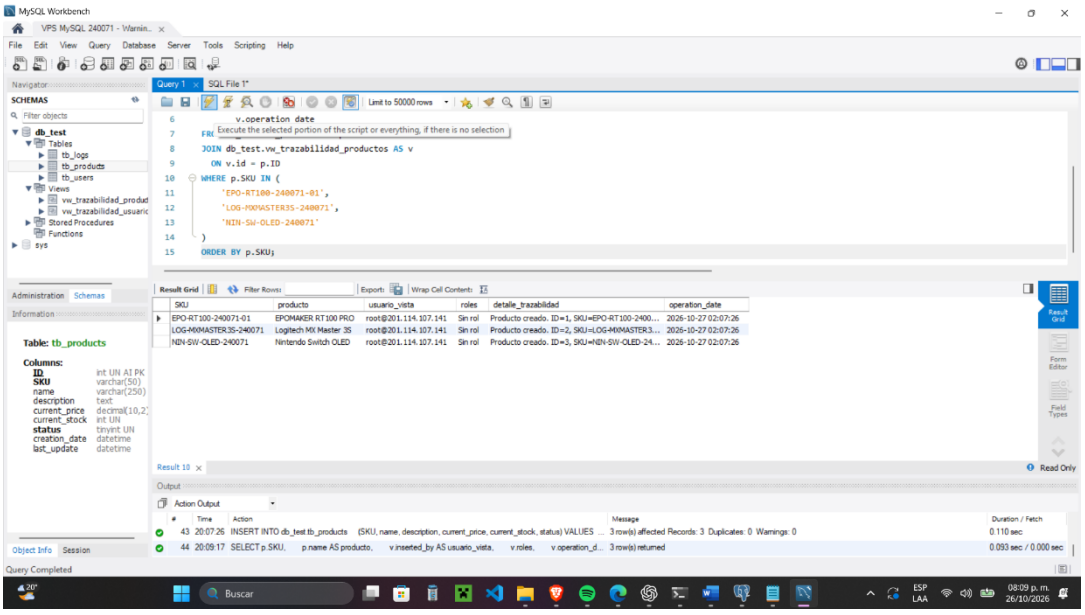
35. Vista de trazabilidad en MySQL. Objetivo: consultar de forma legible las operaciones auditadas. Proceso:



se creó y consultó la vista que organiza los registros de tb_logs para usuarios y productos. Resultado: Workbench mostró la vista y sus datos asociados; conservar la consulta con filas de ejemplo como evidencia.

36. Consistencia de objetos y trazabilidad. Objetivo: comprobar que tablas, triggers y vistas entregan datos coherentes. Proceso: se consultaron catálogos y bitácoras y se insertaron/modificaron/eliminaron registros de prueba. Resultado: las capturas muestran eventos Create, Update y Delete; cuando la consulta de la vista aparece vacía, se requiere una operación posterior y una nueva consulta para probar su funcionamiento.



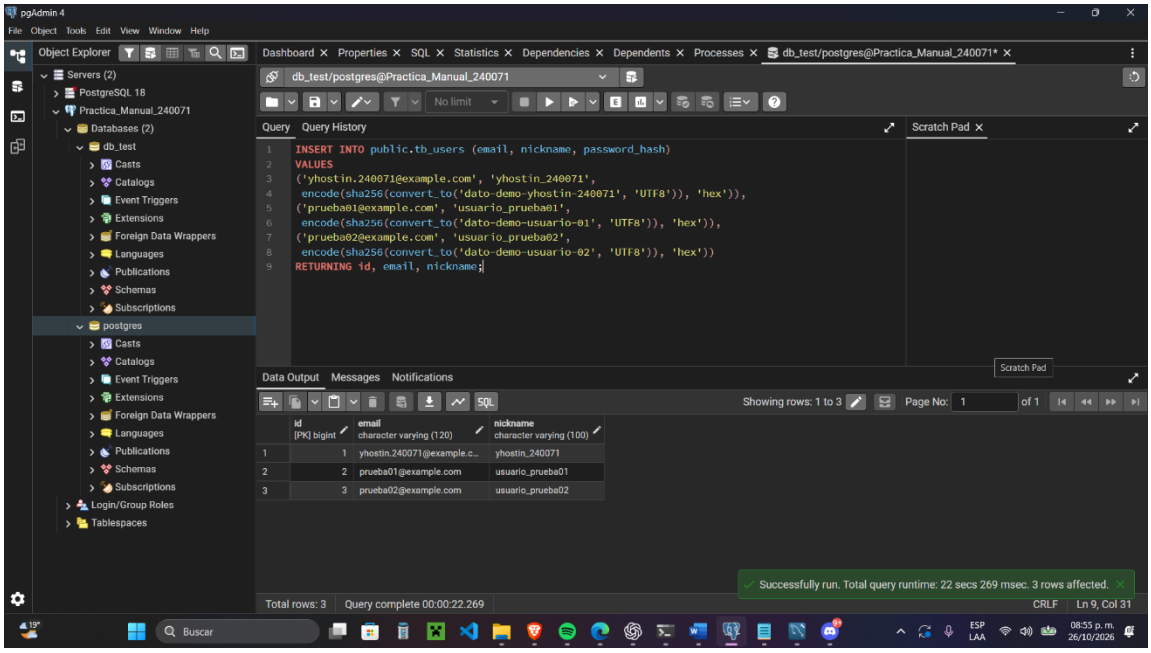


F. Ejecución de los casos de prueba

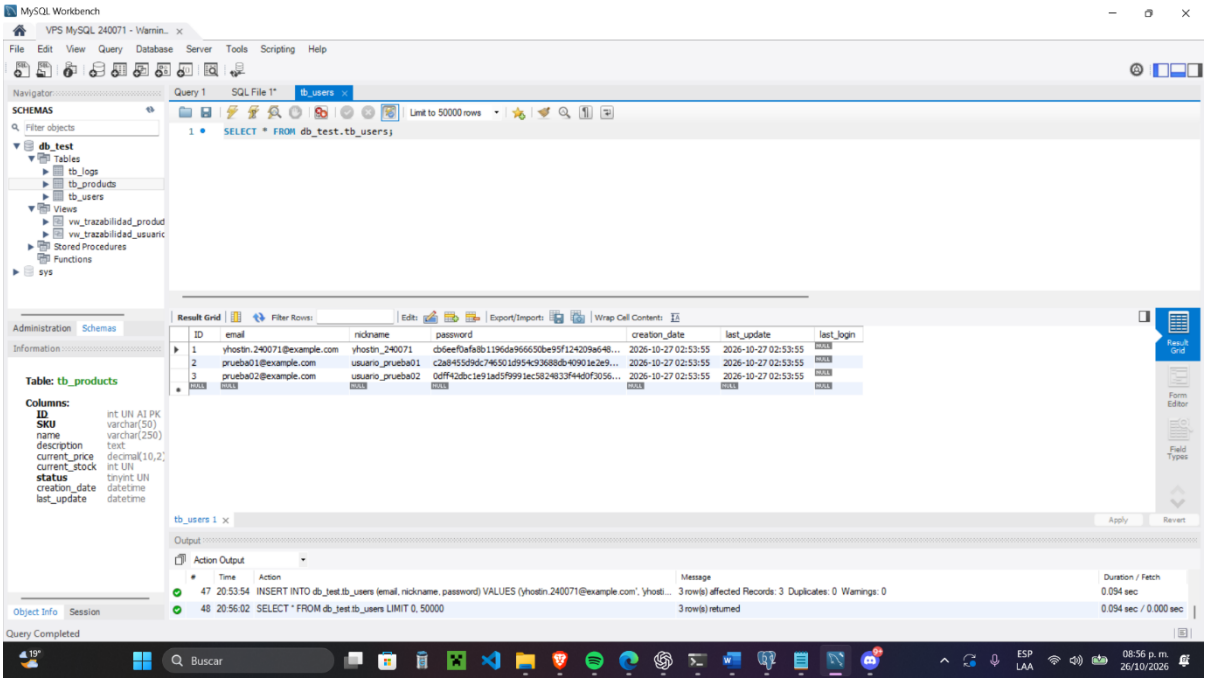
37. Entrega de accesos para pruebas. Objetivo: permitir que los participantes ejecuten los casos con sus cuentas y desde clientes externos. Proceso: se usaron cuentas de prueba en las herramientas cliente y se omitieron las contraseñas del reporte. Resultado: la evidencia de un canal autorizado de entrega debe agregarse sin exponer secretos; hasta entonces, este criterio queda pendiente de comprobación.

38. Caso de prueba 01: alta de tres usuarios por la cuenta administradora.

Objetivo: evaluar la inserción de tres usuarios con privilegios administrativos, incluyendo al estudiante. Proceso: se ejecutó INSERT sobre tb_users con la cuenta administradora de cada motor y luego SELECT

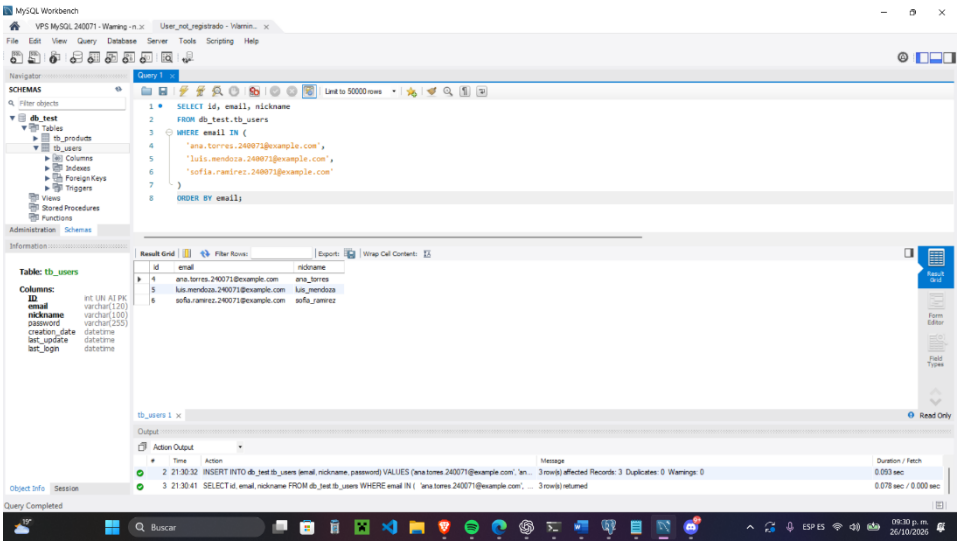


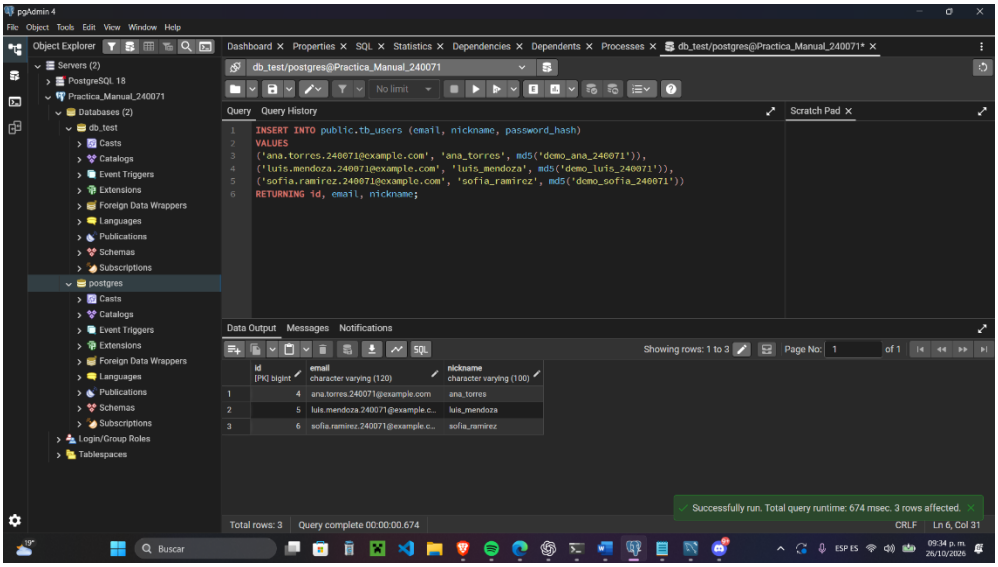
para revisar las filas. Resultado: se muestran los registros de prueba y el usuario de matrícula 240071.



39. Caso de prueba 02: alta de tres usuarios con la cuenta de usuario no registrado.

Objetivo: probar la inserción con el perfil limitado not_registered. Proceso: se conectó como usuario_no_registrado en MySQL Workbench y se insertaron tres filas en tb_users; después se consultaron los registros. Resultado: las capturas muestran las tres altas. El permiso en tb_users se habilitó para la prueba, así que describirlo como cuenta limitada, no como ausencia total de rol.



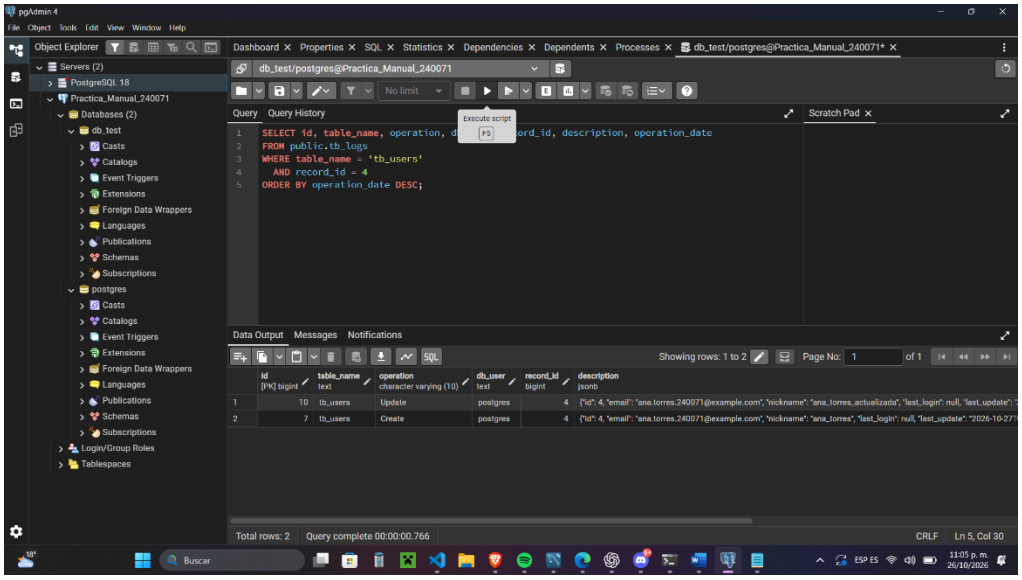


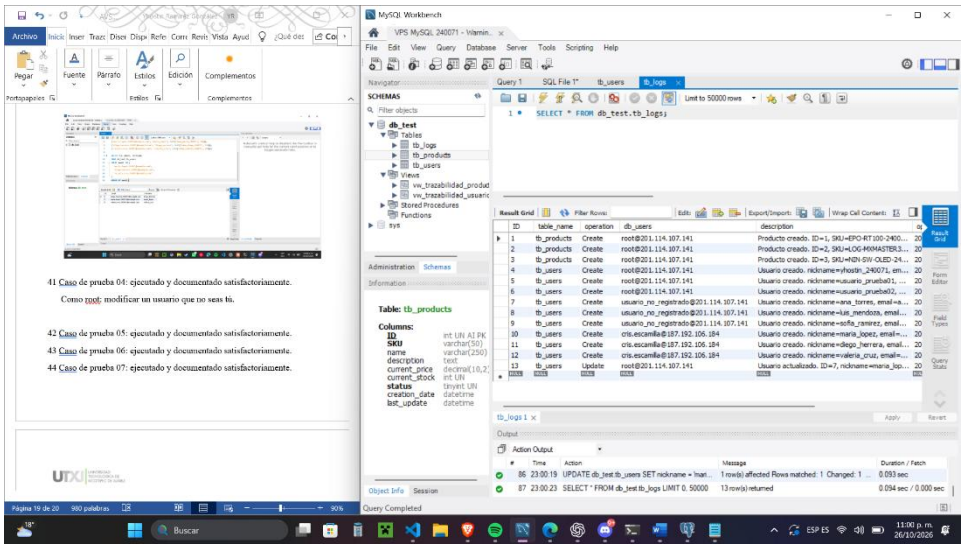
40. Caso de prueba 03: alta de tres usuarios como support.

Objetivo: evaluar la inserción de tres usuarios con el rol support. Proceso: se intentó documentar la operación desde PostgreSQL, pero la captura muestra que Query Tool estaba conectado como postgres, no como cris.escamilla. Resultado: el caso no está comprobado con sesión support; repetirlo usando la cuenta y guardar captura de usuario conectado y filas insertadas.

41. Caso de prueba 04: modificación de un usuario por la cuenta administradora.

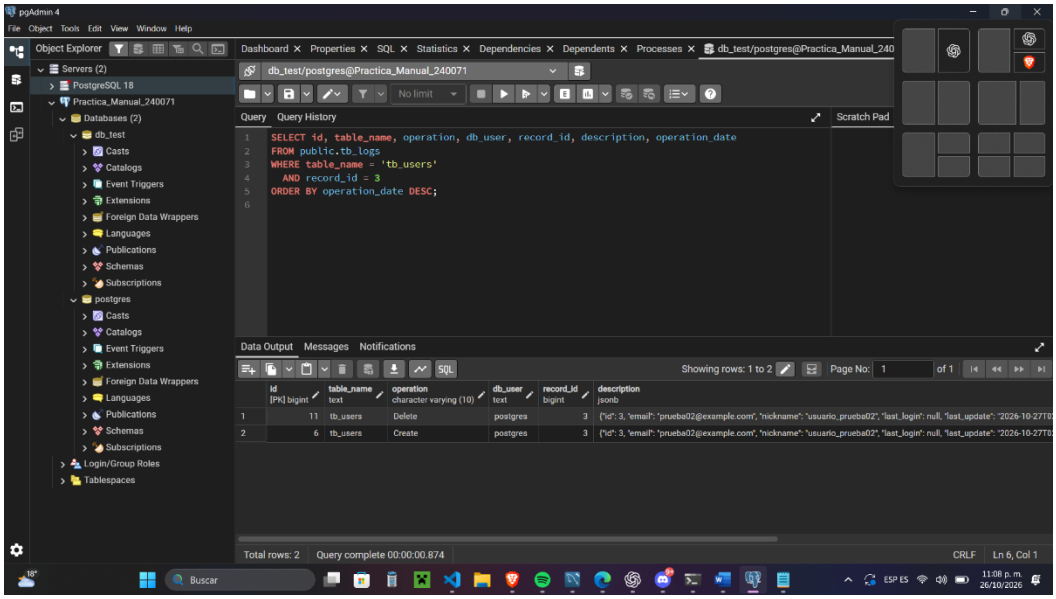
Objetivo: comprobar que la cuenta administradora puede modificar un usuario distinto del estudiante. Proceso: se ejecutó UPDATE sobre el registro de prueba y se consultó nuevamente la fila. Resultado: el valor actualizado aparece en la consulta; la operación debe quedar visible también en la bitácora.

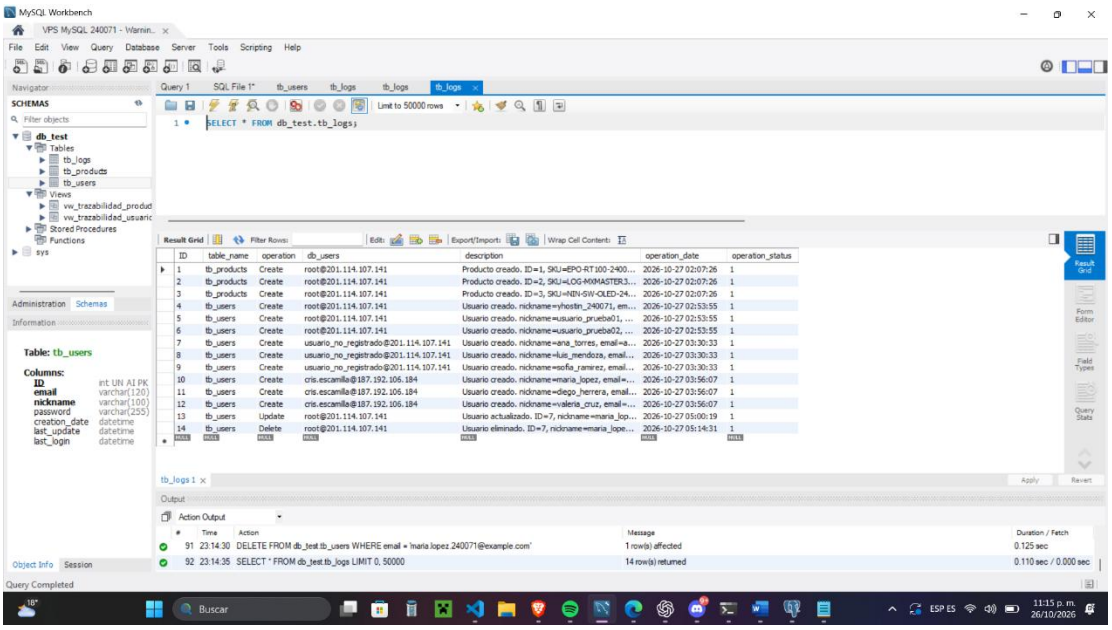




42. Caso de prueba 05: eliminación de un usuario por la cuenta administradora.

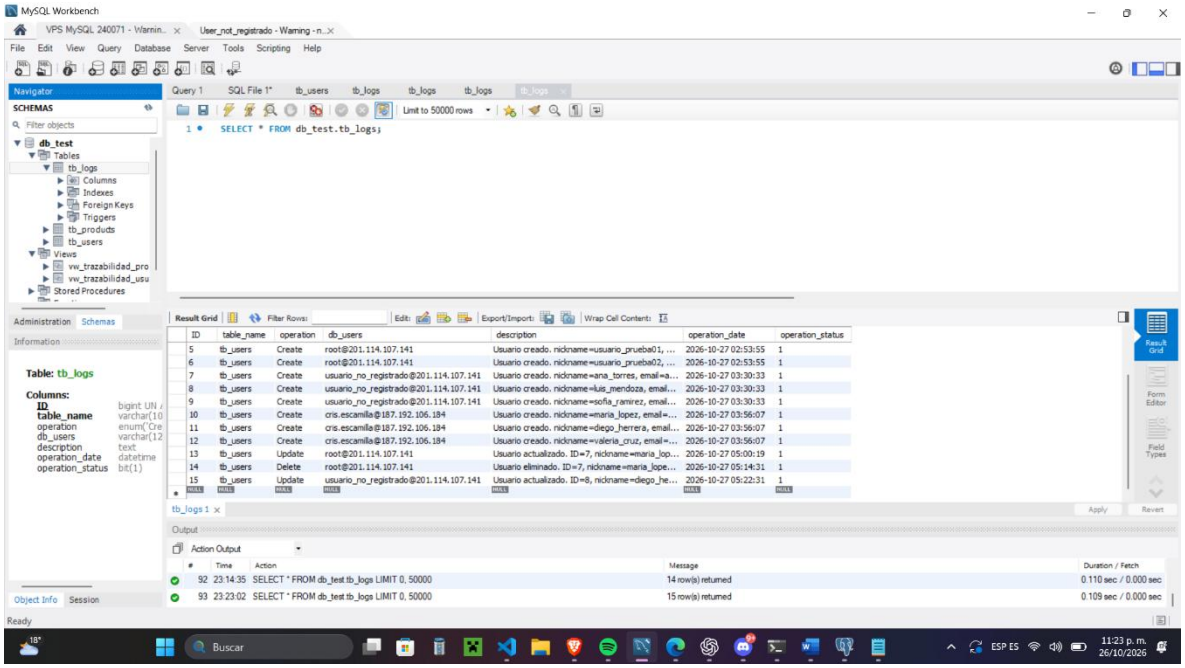
Objetivo: verificar la eliminación de un registro de prueba por la cuenta administradora. Proceso: se ejecutó DELETE y luego SELECT para confirmar que el usuario ya no existía. Resultado: la ausencia del registro y el evento de trazabilidad documentan la baja.





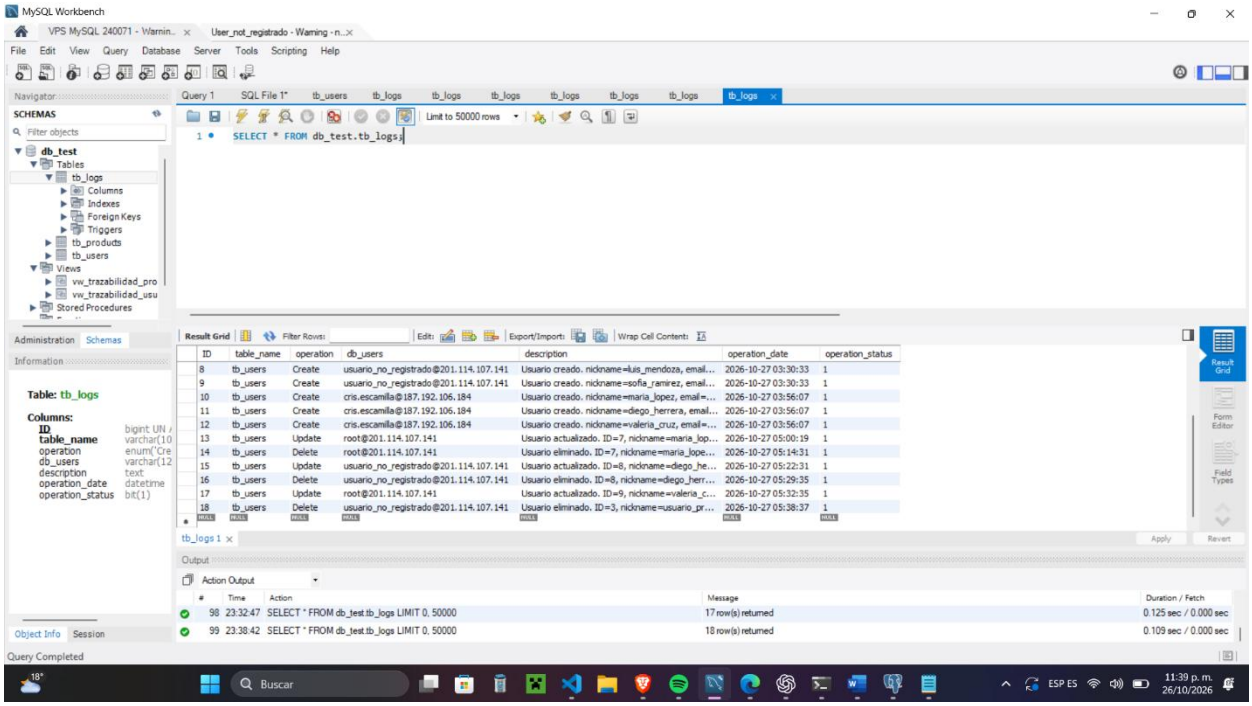
43. Caso de prueba 06: modificación de un usuario con la cuenta de acceso limitado.

Objetivo: evaluar la modificación de un usuario con la cuenta de acceso limitado. Proceso: usuario_no_registrado ejecutó UPDATE sobre un registro de prueba; después se consultó su valor. Resultado: el cambio aparece en la consulta y corresponde a la cuenta limitada usada en la prueba.



44. Caso de prueba 07: eliminación de un usuario con la cuenta de acceso limitado.

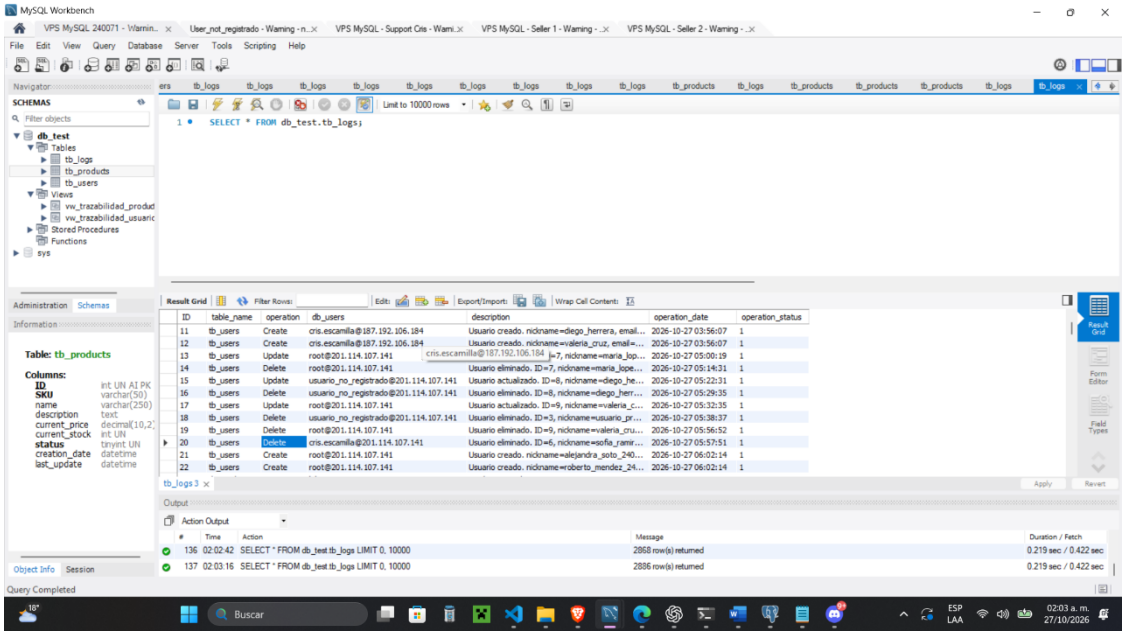
Objetivo: comprobar si el perfil limitado puede eliminar usuarios según los permisos asignados. Proceso: se intentó la baja del registro y se revisó su ausencia y la bitácora. Resultado: el borrador indica que la



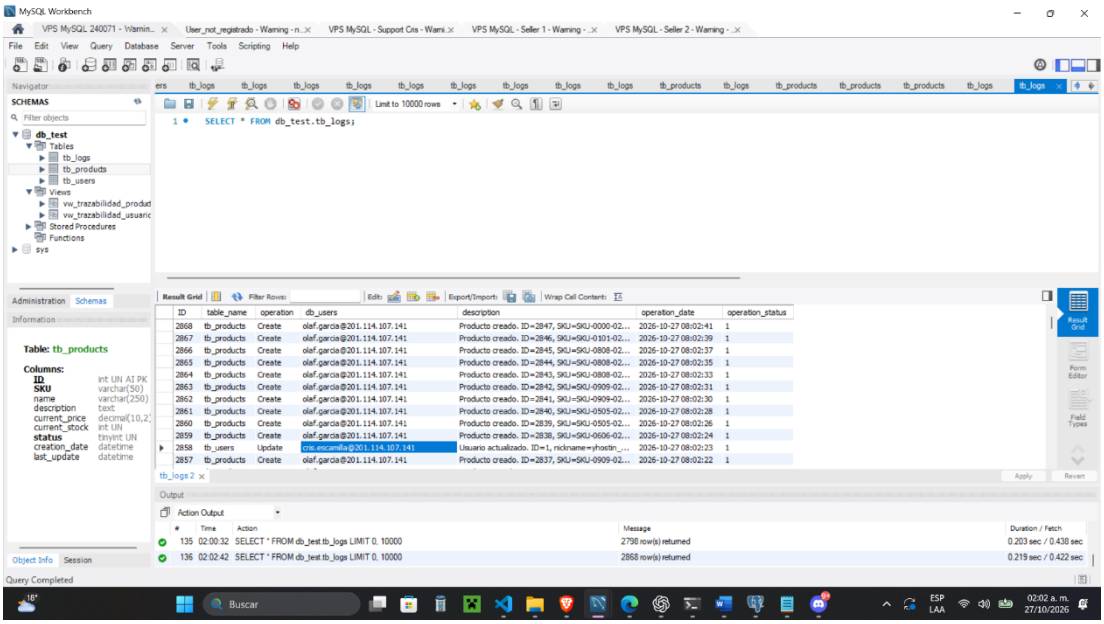
operación fue permitida, pero debe conservarse una captura legible que identifique cuenta, sentencia y resultado para verificar el criterio.

45. Caso de prueba 08: modificación y eliminación de usuarios como support.

Objetivo: probar modificación y eliminación de usuarios con el perfil support. Proceso: las capturas muestran

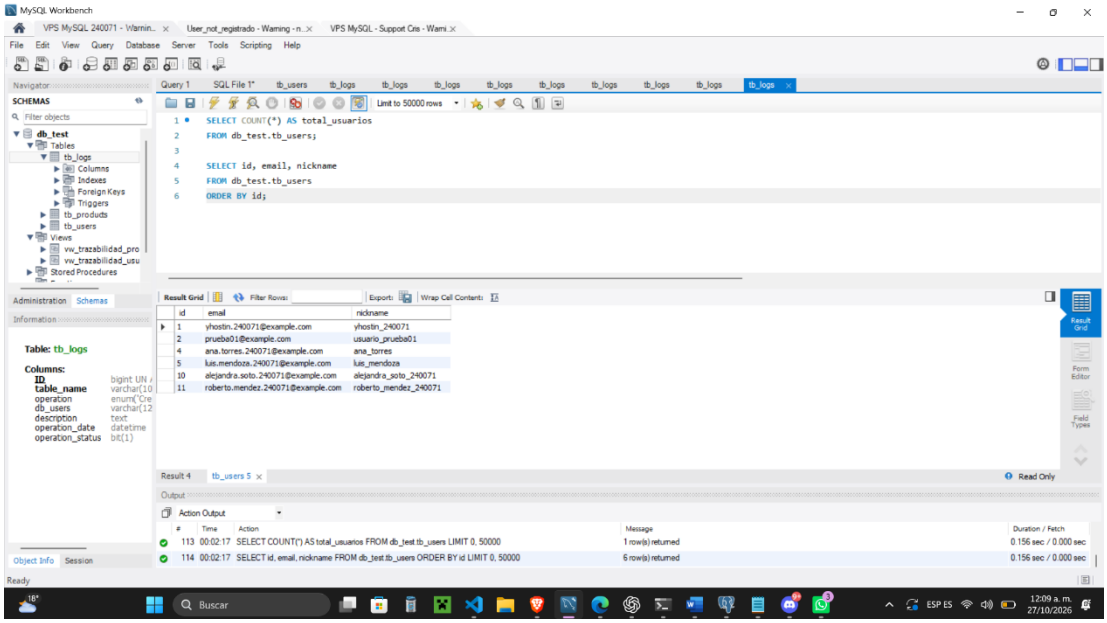


consultas y resultados en MySQL Workbench.



46. Caso de prueba 09: verificación de seis usuarios.

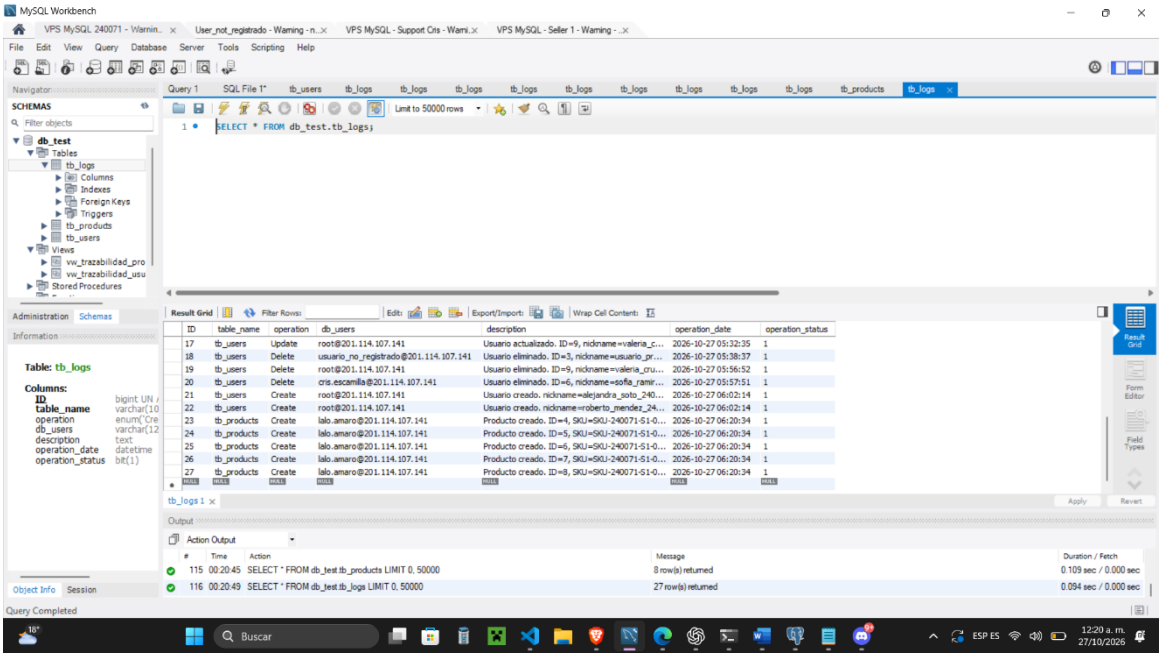
Objetivo: validar el total requerido al concluir las operaciones de usuarios. Proceso: se ejecutó SELECT



COUNT(*) y se consultó la lista de usuarios en db_test.tb_users. Resultado: MySQL Workbench mostró seis registros; conservar juntos el conteo y el listado como evidencia.

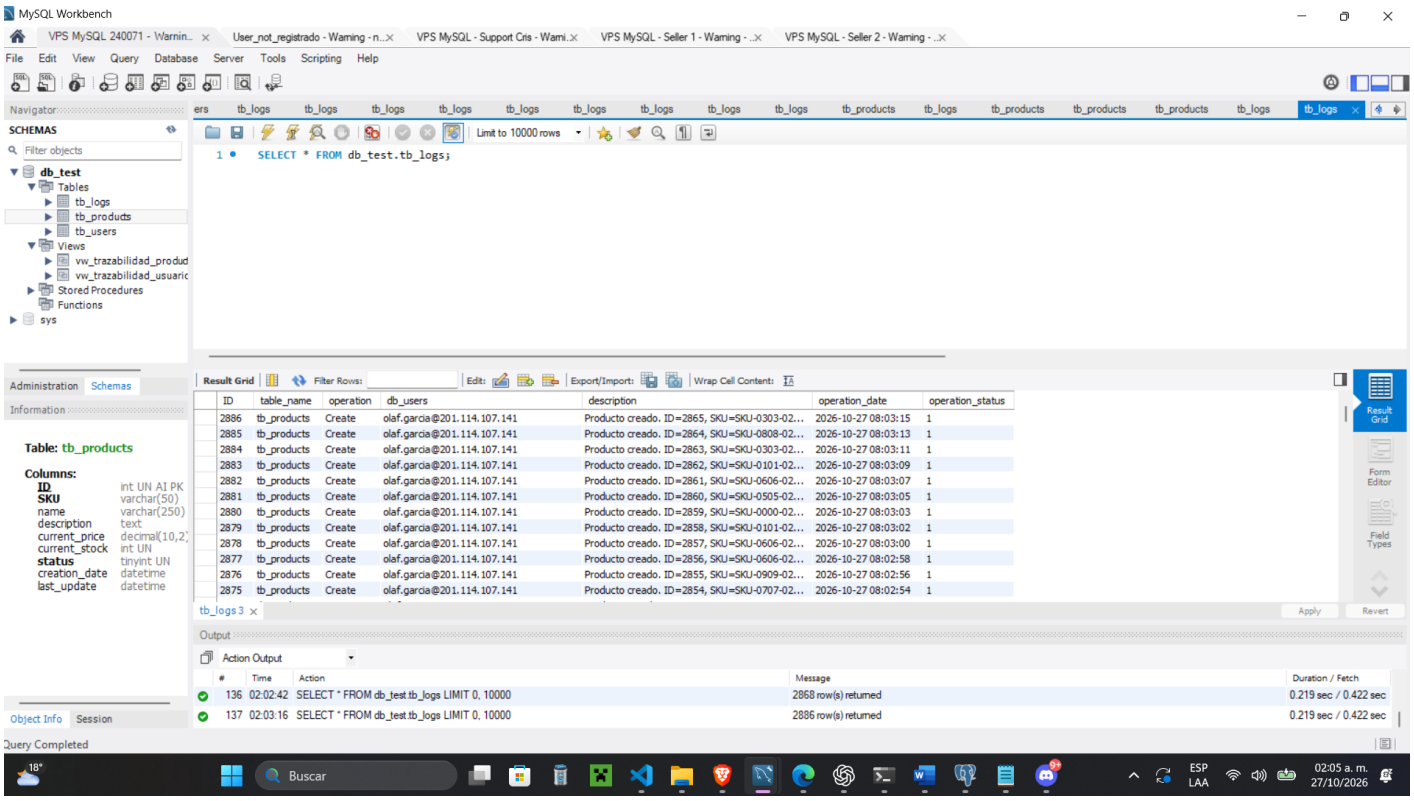
47. Caso de prueba 10: operaciones de productos de los pasos 11, 12 y 13.

Objetivo: probar las inserciones de productos de administrador y Seller 1. Proceso: el administrador agregó



dos productos y Seller 1 insertó cinco productos; se consultó tb_logs. Resultado: la captura de trazabilidad muestra eventos Create de ambas operaciones.

Objetivo: importar 5,000 productos con Seller 2. Proceso: se preparó un CSV de 5,000 filas y se seleccionó la tabla existente db_test.tb_products en el asistente de importación. Resultado: el asistente estaba configurado, pero falta la ejecución final y una consulta COUNT(*) que confirme 5,000 filas; el paso sigue pendiente de evidencia.



G. Reporte técnico individual y control de versiones

En esta sección se presenta el estado del reporte individual y de los entregables de control de versiones. Cada indicador se describe con su objetivo, proceso y resultado comprobable.

48. Portada.
Objetivo: identificar formalmente la institución, la asignatura, la práctica y al estudiante.
Proceso: se incorporaron programa educativo, nombre, matrícula, grupo, equipo y fecha; se dejó el campo docente para completar.
Resultado: la portada contiene los datos disponibles y requiere completar el nombre del docente.

49. Índice.
Objetivo: permitir ubicar cada capítulo y subcapítulo.
Proceso: se organizó el índice según las secciones desarrolladas y se verificaron las páginas en la versión final.
Resultado: los títulos y números de página corresponden a la paginación actual.

50. Descripción de la práctica.
Objetivo: explicar el contexto, el propósito, las tecnologías y el alcance.
Proceso: se resumieron VPS, Docker Compose, PostgreSQL, MySQL, herramientas cliente, roles, objetos y

pruebas.
Resultado: la sección presenta objetivo y tecnologías utilizadas.

51. Desarrollo técnico.
Objetivo: documentar procedimientos, configuración, comandos y resultados.
Proceso: se organizaron los pasos de VPS, cuentas del sistema, herramientas, contenedores, puertos, conexiones, privilegios y objetos SQL.
Resultado: el desarrollo queda descrito por etapas; los indicadores pendientes se señalan en lugar de reportarlos como completos.

52. Conclusiones personales.
Objetivo: reflexionar sobre competencias, dificultades y resultados.
Proceso: se redactó una conclusión sobre despliegue, acceso remoto, permisos, trazabilidad y problemas de autenticación.
Resultado: se identifican aprendizajes y tareas aún necesarias para completar la evidencia.

53. Capturas de pantalla.
Objetivo: demostrar las etapas y resultados relevantes con evidencia legible.
Proceso: se conservaron las capturas de pantalla completa del borrador y se ubicaron junto a los puntos relacionados.
Resultado: las imágenes originales permanecen sin modificación; agregar las capturas pendientes de support, importación CSV y control de versiones.

54. Rama de trabajo.
Objetivo: trabajar en la rama privada Manual_VPS_BDs_Relacionales.
Proceso: crear o confirmar esa rama en el repositorio privado y capturar su nombre y estado.
Resultado: el borrador no contiene evidencia de creación/uso de la rama; indicador pendiente.

55. Archivos y commits.
Objetivo: incorporar los entregables requeridos y registrar los cambios.
Proceso: añadir Reporte_Unidad1_240071.pdf, docker-compose.yml, sql/postgresql.sql, sql/mysql.sql y README.md; crear commits sin secretos.
Resultado: los commits y archivos deben añadirse al repositorio y documentarse; todavía no hay evidencia adjunta.

56. Fusión a rama principal.
Objetivo: integrar el trabajo en main o master sin conflictos pendientes.
Proceso: fusionar la rama Manual_VPS_BDs_Relacionales y revisar el estado final del repositorio.
Resultado: la fusión no está demostrada en el borrador; agregar evidencia real después de completarla.

7. Conclusiones personales

En esta práctica desarrollé habilidades para desplegar bases de datos en un VPS, administrar servicios aislados con Docker Compose y conectarme desde clientes gráficos. También practiqué la creación de roles, la asignación de privilegios y la verificación de operaciones mediante consultas y trazabilidad. Las dificultades principales estuvieron en la configuración de puertos, las reglas de acceso y la autenticación de cuentas con roles; las atendí revisando permisos, conexiones y resultados SQL. Concluyo que una base de

datos en la nube requiere controlar tanto su disponibilidad como quién puede conectarse y qué operaciones puede realizar. Antes de entregar, debo completar y documentar la autenticación de support en PostgreSQL, el procedimiento de borrado de support en MySQL, la importación del CSV de Seller 2 y la evidencia de commits y fusión a la rama principal.